Verifiable On-Board Computation Under Adversarial Manipulation in CubeSats

Sondre Rønjom

***Abstract*—Small satellites increasingly process sensor data on board and downlink only a derived product. A conventional signature authenticates the acquisition, but nothing establishes that the derived product was computed from it by the intended algorithm once the flight software is compromised. Succinct proofs of computation close this gap in principle: the satellite emits a product d and a proof π that $d = A_\theta(D)$ for the authenticated acquisition D, and the ground verifies π without D. This paper asks whether the prover cost of such proofs is compatible with a CubeSat. We survey the relevant proof systems, provenance mechanisms and roots of trust, set out the adversary model, and then report a complete proof of concept for a synthetic aperture radar (SAR) ship-detection chain: raw echo, range-Doppler focusing, intensity, CFAR detection, connected components and candidate chips, all in bit-exact fixed-point arithmetic. We implemented the same chain under two arithmetizations, a Boolean circuit over $\mathbb{F}_2$ proved with Flock and a multi-table AIR over the Goldilocks field proved with a Plonky3 batch-STARK, and measured both. The STARK route proves the full 512×1024 chain bit-exactly, binds the raw acquisition into the statement by an in-circuit Poseidon2 digest equal to a signed capture record, reaches 113.3 proven bits at the provenance leaf and 111.4 in the recursive aggregate, rejects every attack in the recorded hostile batteries, fits a 4–8 GiB memory envelope through segmentation, and aggregates the segment proofs recursively into one externally verified proof. The proof of concept also locates precisely what separates this result from a flight system: the proof is still 1.14–1.30$\times$ the raw bytes it certifies, leaf proving runs at about 2 MiB per 224 s on a laptop, and hierarchical aggregation is not yet semantically complete. Each gap has an identified lever, and we close with a roadmap from laboratory measurement to on-board flight testing.**

***Index Terms*—Verifiable computation, STARK, zero-knowledge proofs, CubeSat, synthetic aperture radar, provenance, trusted platform module, on-board processing.**

I. Introduction

Small satellites, including CubeSats, operate under tight constraints on computation, memory, energy and communication bandwidth. These constraints also limit the mechanisms that can be deployed to protect on-board computation against cyberattacks. In this work we consider techniques for providing *computational integrity*: a ground user should be able to verify that sensor data received from a satellite is the correct result of an intended on-board computation, even if the software performing that computation is untrusted or has been maliciously modified.

On-board computation is attractive because it can substantially reduce downlink requirements. Rather than transmitting an entire sensor acquisition, the satellite may process the data locally and transmit only the information relevant to the user. This creates a gap between *authentication* and *computation*. A trusted hardware component, such as a TPM or a similar hardware root of trust, can authenticate an acquired sensor data set D together with associated metadata. Once D is transformed by an on-board processing algorithm, a conventional digital signature on D does not by itself establish that the derived data was computed correctly.

This motivates verifiable computation. Let A_θ denote a publicly specified signal-processing algorithm with parameters θ, and let d denote the derived product transmitted to the ground. The desired statement is $d = A_\theta(D)$ where D is an authenticated acquisition. A succinct proof π lets the ground station verify this relation without re-executing the computation and without receiving D. The verifier checks

$$\exists D: \quad D \text{ is the authenticated acquisition } \wedge\ d = A_\theta(D). \tag{1}$$

The mechanism binding the witness D to the hardware-attested acquisition is a system-design question that we treat separately from the cost of proving the signal processing itself. Depending on the proof system and its parameters, an adversary's probability of causing an incorrect computation to be accepted can be made cryptographically small, of the order of 2^{-100} under the corresponding soundness and cryptographic assumptions.

A. Motivating example: SAR ship detection

Consider a satellite carrying a synthetic aperture radar for maritime surveillance, for example detection of vessels engaged in unreported fishing (Fig. 1). A ground user requests observations of an ocean region and requires assurance that the received data is authentic. Transmitting the complete SAR product to every user is prohibitively expensive, so the satellite executes an on-board chain that extracts candidate detections and small image chips: $d = A_\theta(D)$ with $|d| \ll |D|$. The reduction creates the integrity problem. Malicious software could alter detections, fabricate targets or, most importantly for surveillance, suppress detections before downlink. Authenticating the transmitted detections does not show that further detections were not omitted. A proof of computation supplies the missing link: the ground verifies that d is exactly the certified processing algorithm applied to the authenticated acquisition, a form of cryptographically verifiable provenance from sensor to product.

The principal question addressed here is not whether such proofs exist in theory but whether their *prover cost* is compatible with a resource-constrained satellite. Proof generation must occur on board, where memory, processing power and

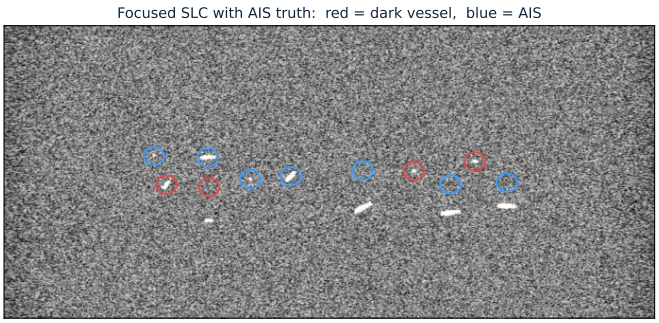


Fig. 1. The simulated scene used throughout: focused SLC with ground truth, 512 pulses by 1024 range samples, sea state 3. Blue circles are AIS-reporting vessels, red are "dark" vessels; the bright streaks below the circles are moving targets displaced in azimuth by $v_r R_0/V$.

energy are limited, whereas verification is performed by a substantially more powerful ground station.

B. Proof systems and arithmetization

To prove the correctness of a computation, the computation is represented as a collection of algebraic constraints. A valid execution produces a witness satisfying them; an incorrect execution should fail them. Rank-1 constraint systems (R1CS) express computation as quadratic constraints $\langle a_i, z\rangle\langle b_i, z\rangle = \langle c_i, z\rangle$. STARK-style systems use an algebraic intermediate representation (AIR), an execution trace with algebraic transition and boundary constraints. Both encode general computation, but their concrete costs differ substantially with the structure of the workload. This matters for signal processing, since Boolean circuits, word-oriented arithmetic and small-prime-field arithmetic lead to very different proving costs for the same algorithm. Section VI makes that statement quantitative on one workload: the same chain costs 498 committed columns per pixel as a Boolean circuit and 40 to 784 cells per pixel as a prime-field AIR, depending on how its roundings are specified.

C. Contributions

This paper makes the following contributions.

1) A threat model for on-board computation under platform compromise, stating what a proof of computation establishes and what it does not (Section III, Table II).
2) A survey of succinct proof systems, provenance mechanisms and roots of trust against the requirements of a DSP workload on a small satellite (Section IV, Table III).
3) Two complete arithmetizations of the same SAR ship-detection chain, a Boolean circuit over $\mathbb{F}_2$ and a multi-table AIR over a 64-bit prime field, with measured prover cost (Section VI).
4) A provenance construction that binds the raw acquisition into the proved statement by an in-circuit Poseidon2 digest equal to a signed capture record, together with an executable break of the cheaper algebraic-fingerprint alternative (Sections III and VIII).

TABLE I
WORKLOAD LADDER A PRACTICAL SCHEME MUST CLIMB; THE MEASURED SCENE IS BELOW W0.

Class	description	raw volume
this paper	512×1024, 16-bit I/Q, simulated	2.1 MB
W0	one real Sentinel-1-class burst	∼100 MB
W1	25 s dual-pol IW, FDBAQ	∼1.2 GiB
W2	substantial acquisition	10 GiB
W3	wide-area stress case	100 GiB
W4	HRWS ingress, 12 Gbit/s for 60 s	90 GB internal

5) Memory reduction by segmented proving with a joint lookup transcript, and proof compression by recursive aggregation, both measured on the real relation (Section VII).
6) Result tables for proof size, memory, proving and verification time, security and downlink volume (Section VIII), a practicality assessment against mission scale (Section IX) and a research agenda (Section X).

The scope is stated precisely. All measurements are on one simulated 512×1024 scene on a laptop-class machine. The correctness, security and provenance results are complete on that scene. Two practicality targets are not yet met and are reported as measured, the proof is larger than the raw acquisition at every operating point tried, and leaf proving throughput is far from mission cadence. Both are quantified, both have identified levers, and neither is a property of the approach as such.

II. Small Satellites: Constraints and Mission Context

A. The envelope

A CubeSat-class SAR payload sits in a narrow envelope. The prover, if it flies, shares a co-processor with the signal processing itself. The design assumed here is 0.5–8 GB of RAM for the payload computer; 5–10 W available to the prover, against a 6U bus whose radiator caps sustained dissipation near 9 W and a 12U near 21 W; raw acquisitions of 0.5–5 GB per scene and 10–30 GB per day; and downlink as the binding resource. A quad Cortex-A53 class processor draws 1.5–3 W including its DRAM traffic and is therefore time-bound, not power-bound; a fabric accelerator can absorb 3–6 W. Any prover must be judged on peak resident memory, wall-clock per scene, energy per scene, and bytes to the ground, and the four trade against each other.

B. Mission modes

Two modes matter. In *tip-and-cue* the satellite is tasked to image a region and downlinks a product on request; the raw data may be retained and audited later. In *always-on* maritime surveillance the satellite continuously produces detections, the raw data is discarded to make room, and the ground has no opportunity to inspect it. The second mode is where a proof of computation stops being a nicety: there is no other way for the ground to know that a detection was not suppressed. It is also the mode in which throughput, not latency, is the binding

TABLE II
MANIPULATION CLASSES, THE ADVERSARY ABLE TO MOUNT THEM, AND WHAT REJECTS THEM. "LINK" ALTERS THE DOWNLINK ONLY; "ON-BOARD" CONTROLS THE PAYLOAD SOFTWARE; "RE-PROVING" ADDITIONALLY REGENERATES ALL EVIDENCE AFTER THE MANIPULATION. A SIGNATURE-ONLY DESIGN STOPS THE FIRST COLUMN; ONLY THE PROOF OF COMPUTATION STOPS THE THIRD.

Manipulation	adversary	caught without a proof by	caught in the final system by
alter raw bytes after capture	link	capture signature on h_{raw}	same
alter a processing parameter (θ)	link	θ hash in the signed envelope	same; θ is preprocessed in every AIR
alter a published pixel, chip, score	link	product hash in the envelope	lookup argument: every published cell is a bus receive
substitute the acquisition and process honestly	on-board, re-proving	**nothing** (digest recomputed by the prover)	in-circuit $H_{P2}(D) = h_{\mathrm{raw}}$ with h_{raw} from the signed record
self-consistent numeric error (e.g. RCMC index ± 1)	on-board, re-proving	**only full ground replay**	AIR constraint on the interpolation index/weight tables
drop, insert, split or merge a detection	on-board, re-proving	**nothing**; spot check defeated	CFAR, component and grouping AIRs; terminal-sum closure
alter a chip pixel or geolocation	on-board, re-proving	structural consistency only	chip AIR (per-pixel receive); geolocation is verifier-recomputed
omit or splice a segment proof	on-board	–	joint transcript: batch digest, terminal sum, set identity
forge, replace or tamper the capture record	on-board	pinned public key	same; prover refuses without a valid record
refuse to produce a product	on-board	–	not addressed (availability)

quantity, because proving must keep pace with acquisition or the on-board queue grows without bound.

C. Workload scale

For scale, Sentinel-1's published FDBAQ-compressed data rates are 206.6 Mbit/s single-polarization and 413.3 Mbit/s dual-polarization [1]; a 25 s single-pol slice is about 0.6 GiB and 300 s of dual-pol about 14.4 GiB. The S4Pro next-generation concept lists 12 Gbit/s of raw ingress [2]. These are not small-satellite figures, but they bound the workload class a practical scheme must eventually reach (Table I). Native on-board SAR processing itself is credible: DLR's EO-ALERT prototype ran a complete ship-detection chain on FPGA/ARM hardware over a 1,125 km^2 TerraSAR-X scene in about 60 s [3]. That is a processing prototype, not a secure one.

The reference scene throughout this paper is 512 pulses by 1024 range samples of 16-bit I/Q, i.e. a raw acquisition of $2{,}097{,}152$ bytes, two to three orders of magnitude below the smallest operational class. We state that encoding every time a ratio is quoted, because the same proof is $1.07\times$ the raw bytes against u16 samples and $1.43\times$ against the 12-bit ADC depth; the i64 file in which the simulator stores the acquisition is 8.39 MB and is storage, not wire.

III. ADVERSARY MODEL AND THE STATEMENT PROVED

A. Three adversaries

We distinguish three capabilities, in increasing order (Table II). A *link* adversary alters what reaches the ground: raw bytes, parameters, products. A signature over the acquisition digest and the product envelope stops it. An *on-board* adversary controls the payload software after acquisition through a compromised firmware update, a supply-chain implant, a hijacked telecommand link or an untrusted operator. It reads and writes every buffer, runs the processing chain on altered inputs and constructs any downlink message. A *re-proving* adversary is an on-board adversary that additionally regenerates every piece of evidence after the manipulation: it re-hashes, re-signs with whatever keys the software holds, and re-runs whatever receipt mechanism exists. The distinction matters because it separates the mechanisms: signatures and envelopes stop the first; only a proof of computation whose statement is anchored outside the software stops the third. What no adversary can do, by assumption, is alter the acquisition before the trusted capture component has recorded its digest, or forge that component's signature. Every adversary can refuse service; cryptography does not guarantee availability.

B. Manipulations exercised

The concrete manipulations executed against the implementation are listed in Table XVIII: alter an image pixel, a filter tap or a CFAR score; hide a detection; split one vessel into two or merge two into one; alter a published chip; fabricate a group boundary; drop a qualifying candidate; substitute a different raw acquisition and prove the chain honestly on it; corrupt any step of the in-circuit hash of the acquisition; forge, replace or tamper the capture record; splice a segment proof from another run; omit a segment; and claim another segment's legitimate framing. Two of them deserve emphasis. Dropping a qualifying candidate is the surveillance attack: everything transmitted is genuine and only an omission is wrong. Substituting the acquisition and processing it honestly is the provenance attack: every constraint of the chain is satisfied and only the input is wrong. A design that catches the first and not the second, or vice versa, has a hole exactly where the threat lives.

C. What the proof establishes

The statement finally proved is

$$\exists D: \quad H_{P2}(D, \Gamma_{\mathrm{acq}}) = h_{\mathrm{raw}} \ \wedge \ \mathrm{SAR}_{\theta}(D) = Y, \quad (2)$$

where H_{P2} is a Poseidon2 sponge over the canonical 16-bit word stream of the acquisition with a domain-separated

TABLE III
PROOF-SYSTEM AND ASSURANCE FAMILIES AGAINST THE REQUIREMENTS OF ON-BOARD SAR PROVING. VERDICTS ARE OURS, AFTER MEASUREMENT WHERE A MEASUREMENT EXISTS.

Family	What it buys	Verdict for this workload
Multi-AIR STARK + LogUp (Plonky3)	exact 64-bit integer relations; lookups for range checks and wiring; batch commitment; recursion available	**Used.** 2.08 cells per operation; near its floor. Univariate, so cannot host a sumcheck front end.
Boolean R1CS over $\mathbb{F}_2$ (Flock)	native bit operations; XOR free; very fast hash proving	**Measured and rejected** as numeric backend: 498 cols/px, 46 GB at 32×32. Candidate hash co-prover.
GKR layered circuits (Expander)	public-constant linear maps at zero multiplication gates	**Measured, superseded**: Raw-PCS proof $4\times$ raw; no shared commitment with the rounding stages.
Direct FFT / convolution sumcheck (zkCNN)	$O(N)$ prover for linear stages, no butterfly wires	Counted at $80\times$ on the convolution terms; needs a multilinear PCS. **Research, first priority.**
LogUp-GKR	removes lookup helper columns (32.7% of cells)	Pre-registered with thresholds; **not run**.
WHIR / BaseFold / Ligerito (multilinear PCS)	smaller proofs; arbitrary-point multilinear opening	WHIR measured: 20.68 GiB PCS-only, parity with FRI; **not a memory fix**, but the enabler of the sumcheck front end.
Brakedown / Ligero	linear-time, field-agnostic	$\sqrt{n}$ proofs of tens of MB; disqualified on downlink.
Binius64	native shifts and 64-bit multiply	beta; no succinct verification or recursion; ships 96 bits. Addition (210/px) is not primitive.
Circle-STARK (S-two), Hypercube (SP1)	fast 31-bit fields	**Rejected**: chain needs 59–62-bit intermediates.
hc-stark / TinyZKP	16.7 M rows in ≤ 2 GiB	Linux x86-64 only, uni-STARK; **blocked**.
Streaming provers (HOBBIT, Scribe)	bounded RAM	storage traffic $O(C)$; Scribe $\sim$480 B/gate on disk; no SAR demonstration.
Folding / IVC / recursive aggregation	bounded step memory, one final proof	**Used** (Plonky3-recursion); leaf proving remains; tree semantics unfinished.
Pairing / KZG succinct proofs	smallest final evidence	trusted setup, not PQ; does not change the witness workload.
Approximate numerical proofs (Spain)	avoids bit-exact emulation	changes constraint semantics; global tolerance admits local corruption.
Homomorphic MACs / signatures	authenticate restricted linear maps	no coverage of rounding, CFAR, components, completeness.
Spot-check / interactive challenge	detects broad corruption cheaply	**Implemented and demoted**: cannot certify absence of a targeted omission.
Authenticated support + ground replay	exact customer-side check of a requested region	**Recommended first prototype** where a selected-region product suffices.
Trusted native chain (secure DMA, attested bitstream)	removes proving work	**Recommended** where the customer accepts the larger TCB.

header Γ_{acq} (dimensions, format, word count), h_{raw} is a public input of the proof taken from a capture record signed by the trusted capture component, and Y is the published product: the intensity image, the CFAR score and detection mask, the candidate list and the 64×64 chips. Every published array is an individual bus receive from the certified computation, so a single altered pixel in a chip is a lookup-argument failure. The processing parameters θ (chirp, references, RCMC tables, CFAR constants) are preprocessed columns of the AIRs and are therefore part of the statement rather than the witness.

The trust boundary reduces to one rule: *the digest must be fixed at capture, not derived from the witness.* A design in which the prover computes h_{raw} from its own input accepts the substitution attack, because the digest is recomputed from the substituted data and matches by construction. A binding the prover can recompute is not a binding.

D. What the proof does not establish

It does not establish that the sensor was pointed where the metadata claims, that the capture component is genuine, or that the physical scene contained the vessels detected. Detection *completeness* as a physical claim is a detector property (recall 0.65–0.80 in the simulator), not an integrity property; the proof establishes that the certified algorithm, whatever its recall, was applied. Geolocation and AIS correlation are recomputed by the verifier from the certified product rather than proven. And the proof says nothing about availability.

E. Why a signature or an attestation is not enough

A TPM signature on D authenticates the input. It does not constrain the software that transforms D. Remote attestation [4] appraises the state of loaded software, but appraisal of what was loaded is not enforcement of what was executed, and a TPM depends on the components that feed it measurements. A signature on the product d by the payload computer proves only that the compromised computer signed it. The three mechanisms compose: attestation and capture signatures bound the trusted computing base; the proof of computation covers everything outside it.

A cheaper alternative was implemented and rejected: a Fiat–Shamir Merkle spot check over a committed per-cell CFAR witness (I, B, N_T, Q, D), with t challenge cells derived from the commitment root and the product hash. Against an adversary who corrupts a fraction ϵ of cells the soundness is $1 - (1 - \epsilon)^t$: at $t = 128$ that is 128 bits for $\epsilon = 0.5$ but 0.18 bits for $\epsilon = 10^{-3}$, and a single altered cell among 4.2×10^6

TABLE IV
OPTIONS FOR BINDING THE RAW ACQUISITION INTO THE PROVED STATEMENT.

Option	in-proof cost	security	status
algebraic fingerprint $F = \sum r^i D_i$, public r	0 mul gates	**broken**: LLL, ± 21 words, 0.4 s	rejected
B2: raw as the proof's own committed columns	0 (byte equality)	binds to the FRI commitment only	priced (C)
B1: algebraic Merkle / sponge (w=12, counted)	6.6×10^8 cells at 4096^2 (98%)	collision-resistant	priced (C)
B3: SHA-256 in circuit	5.0×10^{10} cells at 4096^2; 2,980 cells/px	standard hash	priced (C), $3.8\times$ chain
Poseidon2 sponge, w16/r12, in the STARK	20.4 cells/px = 1.75%; 29,128 perms	113.3 proven bits	**implemented, measured**
signed capture record (Ed25519) + pinned key	outside the proof	classical signature	implemented (dev key)

The implemented statement is $\exists D:\ H_{P2}(D, \Gamma_{\mathrm{acq}}) = h_{\mathrm{raw}} \ \wedge\ \mathrm{SAR}(D) = Y$ with h_{raw} taken from the signed record, never recomputed by the prover.

TABLE V
SIMULATED ACQUISITION USED FOR EVERY MEASUREMENT.

Parameter	value
carrier / wavelength	9.65 GHz / 3.11 cm
chirp bandwidth / sampling rate	50 MHz / 75 MHz
PRF / platform velocity	1500 Hz / 7500 m/s
altitude / slant range	693 km / 800 km
antenna length (aperture)	24 m (414 pulses)
Doppler bandwidth	1250 Hz
range IRF / PSLR	3.91 m / -41.3 dB (-42.8 theory)
azimuth IRF	10.8 m
range-cell migration	0.48 samples
sea state / clutter / thermal noise	3 / 600 counts / -12 dB
ADC	16-bit I/Q, scale 12 (peak code 94,759)
scene	512×1024, 10 vessels, seed 20260814
vessel kinds (RCS)	skiff 4–11, fishing 12–20, trawler 16–24, cargo 21–30 dB
dark / moving fraction	0.40 / 0.35

TABLE VI
FIXED-POINT SPECIFICATION OF THE CERTIFIED CHAIN (ALL VALUES EMBED IN GOLDILOCKS, $p = 2^{64} - 2^{32} + 1$).

Quantity	format	shift after multiply
raw sample	int16	–
conditioned sample	43-bit signed	DC 16, ADC 20
FFT twiddle	17-bit Q1.15	15, then 1 per stage
range reference H_r	19-bit	16
azimuth reference H_a	19-bit	17
RCMC weight (2 taps)	17-bit Q1.15	15
SLC quantiser	31-bit saturating	–
intensity gain	$885{,}443{,}716 \gg 57$	57
intensity code	32-bit, clip $2^{32} - 1$	–
integral image / CFAR Q	$\le 2^{58}$, Q 40-bit sign witness	–
component distance / group	20-bit / 20-bit	–
rounding rule	$\mathrm{rs}(x, k) = (x + 2^{k-1}) \gg k$	

passes with probability about $1 - 3 \times 10^{-5}$. A suppression attack that erases the brightest candidate, reprocesses honestly and re-proves is accepted. Sampling cannot certify the absence of a small targeted omission; it is retained in the demonstrator only as a labelled non-authoritative row.

IV. BACKGROUND: SUCCINCT PROOFS, PROVENANCE AND ROOTS OF TRUST

A. Arithmetizations

1) R1CS and GKR: An R1CS instance over a field $\mathbb{F}$ is a system $\langle a_i, z\rangle\langle b_i, z\rangle = \langle c_i, z\rangle$ over a witness vector z that includes the public inputs, one constraint per multiplication. GKR-style systems [5] prove a layered arithmetic circuit by the sumcheck protocol [6], reducing a claim about the output layer to a claim about the input layer one layer at a time, so that the prover never commits to internal wires. Linear layers with public coefficients are especially cheap: a multiplication by a constant is a scalar in the layer's linear map, not a gate, which is why an N-point NTT with public twiddles compiles to zero multiplication gates.

2) AIR and STARK: An AIR [7] represents a computation as a $W \times H$ execution trace. Each column is a polynomial of degree less than H over a multiplicative subgroup of the field; constraints are low-degree polynomial identities between a row and the next row, plus boundary conditions. The prover commits to the columns, and proves that the constraint polynomial vanishes on the trace domain by exhibiting the quotient by the vanishing polynomial and testing the quotient's low degree with FRI. The cost is governed by the number of committed cells $W \times H$, the constraint degree (which sets the quotient's size) and the blowup factor of the low-degree extension. As a worked example, the butterfly AIR that makes up two thirds of our chain has main columns $(a_{re}, a_{im}, b_{re}, b_{im}, t_{re}, t_{im}, o^+, o^-, \ldots)$, preprocessed columns for the twiddle (w_{re}, w_{im}) and four addresses, and constraints

$$b_{re} w_{re} - b_{im} w_{im} + 2^{14} = t_{re} 2^{15} + r_1, \tag{3}$$

$$a_{re} + t_{re} + 1 = o^+_{re}\, 2 + r_2, \tag{4}$$

and the like, with each remainder r_j range-checked. Everything else in the stage is dataflow.

3) Lookups and buses: A lookup argument proves that every row of one column set appears in another. LogUp [8] does it with logarithmic derivatives: for a random challenge β and per-row fingerprints $f_i = \langle \alpha^{(0)}, \alpha^{(1)}, \ldots\rangle \cdot (v_i)$ of the tuples, the prover shows

$$\sum_{\text{sends}} \frac{m_i}{\beta - f_i} \;=\; \sum_{\text{receives}} \frac{1}{\beta - f_j}, \tag{5}$$

with multiplicities m_i, by a running-sum column per bus whose final value (the *terminal*) must be zero over the whole batch. A *bus* is a named lookup on which any table may send or receive; with the tuple $(\text{address}, \text{re}, \text{im})$ it turns dataflow into a multiset equality that the verifier checks and the prover

TABLE VII
PRESCREENER STUDY: THE QPSK-QUANTISED RANGE FILTER (Q1024) AGAINST THE FULL-PRECISION REFERENCE (REF), AND THE SYSTEM-LEVEL CLOSURE. PAIRED EXACT MCNEMAR WITH HOLM CORRECTION.

Question	result	status
Q1024 vs REF, pooled (4,800 vessels)	b=c=89, ΔR=0.000, p=1.0	no detectable difference
Q1024 vs REF, land edge	ΔR= − 0.040 (p=0.004), −0.053 (p=0.007)	deficit, survives Holm
cause of the coastal deficit	taper removal, not phase quantisation	F0→F1→F2 decomposition
extraction geometry (patch 64→128)	recall 0.268 → 0.516 at $\eta \times 1.12$	dominant recall loss
hard per-scene budget, caps 1/3/5/10%	worst η = cap; utilisation 0.85–0.93	never violated
pooled recall at η=3% (CFAR-max key)	0.406; truth-aware oracle 0.551	scheduler headroom real
scheduler / taper repairs of the coast	all rejected (de-normalisation 0.25 → 0.023)	–
compute, coarse FFT + fine at η=3%	0.70× HQ chain (5.1× as direct FIR)	no compute saving
downlink / RAM at η=3%	204× / 720–1440×	established
tap sweep, pooled recall (η=5%)	L=64: 0.281, 128: 0.350, 512: 0.469, 1024: 0.505	$C_{\mathrm{dir}}/C_{\mathrm{HQ}}$ 0.32 → 5.10

Coefficient simplicity buys nothing in a prime-field STARK (twiddles are preprocessed constants), so this study does not bear on proof cost; its residue is the selectivity bound (∼ 3×, not 6–18×).

TABLE VIII
THE SAME 512 × 1024 CHAIN UNDER DIFFERENT ARITHMETIZATIONS. COMMITTED SIZE IS PER PIXEL; PROOF BYTES PER PIXEL AT THE STATED PROVER MEMORY.

Arithmetization	committed/px	proof B/px	status
Boolean R1CS, $\mathbb{F}_2$ (Flock), 4 GB	498.3 cols	16.8	meas.
AIR, bit-column range checks	2,115 cells	6.80	meas.
AIR, LogUp range table	784 cells†	4.69	meas.
AIR, exact integer convolution	40 cells	0.81	(C)

†1,165 cells/px including lookup columns; 811 with provenance tables.

TABLE IX
AND-GATE COSTS OF THE BOOLEAN ARITHMETIZATION (MEASURED ON BUILT CIRCUITS; XOR IS FREE).

Gadget	ANDs	alternative
n-bit add / sub / mux / eq / lt	$n-1$ / n / n / $n-1$ / n	optimal
round-shift by k of n bits	$n-k$	
16 × 16 unsigned multiply	496	floor 480
twiddle multiply, NAF constant	879	5,764 general
butterfly (43-bit, 17-bit twiddle)	1,137	1,842 binary recoding
complex multiply (Karatsuba)	4,783	6,026 schoolbook
matched-filter bin (constant)	967	6,448 witness coeff.
RCMC interpolation (constant idx)	530	2,986
intensity $\lvert z\rvert^2$ + gain + clip	1,950	2,940
CFAR cell (amortised, 64 × 64)	229	1,772 witness form
connected components / px (8-conn)	$8L + 8D + 20 \approx$ 195–340	
SHA-256 compression	22,573	Bristol-equivalent
whole chain / px, 8 × 8 → 64 × 64	10,175 → 19,453	

TABLE X
PROVER MEMORY BY PHASE AND PER-PIXEL CELL BUDGET (UNSEGMENTED, 59 TABLES, KECCAK).

Phase	peak GiB
trace generation	4.71
preprocessed commit	8.44
main-trace LDE	16.17
permutation	20.81
quotient + FRI	22.95

Cell class	cells/px	share
shifts / range checks	518	44.4%
wiring (LogUp)	382	32.7%
data values carried	154	13.2%
compare/label/aggregate	112	9.6%
total	1,165	

Commitment is 79% of peak; memory tracks trace cells at ∼40 GiB per 10^9 cells. Butterfly tables are 66.5% of cells (40 of 59 tables).

TABLE XI
COUNTED PER-PIXEL BUDGET OF THE EXACT-CONVOLUTION CHAIN (NO AIR BUILT), AND BANDED PROVER COST ON A 4× CORTEX-A53 AT 1.2 GHZ FOR A 2048^2 SCENE AT BLOWUP 4, 60 QUERIES.

Stage	cells/px	aux	lookups/px	sumcheck ops
conditioning	6	2	4	4
range compression (linear)	6	4	6	2
RCMC (linear)	0	0	0	2
azimuth compression (linear)	4	2	4	2
intensity $\lvert z\rvert^2$, gain, clips	18	13	14	1
CA-CFAR	6	4	4	5
total	40	25	32	16

prover, 2048^2: butterflies 1.27×10^{10}, hashed 10.7 GB, 44–110 s, 0.4–1.1 kJ
as-built rounding FFT, same scene: 469–1202 s; 4096^2: 2019–5175 s (infeasible)
overhead of verifiability vs. computing A_θ by NTT: 31–189×

cannot bend. The price is the helper columns: one running-sum column and its auxiliaries per bus per table, which are 32.7% of our cells. LogUp-GKR [9] proves the same sum by a GKR tree and commits only the multiplicities.

4) Boolean R1CS: Flock [10] specialises R1CS to $\mathbb{F}_2$ in the form $(Az) \odot (Bz) = z$ with $C = I$, so that only AND gates are committed, XOR is free (it becomes a row of A or B), and the witness is a bit vector of length 2^m proved by a zerocheck and a block-diagonal lincheck over $\mathbb{F}_{2^{128}}$ and committed by ring-switching into a Ligerito [11] multilinear commitment. It is the natural home of hash functions; whether it is the home of fixed-point DSP is what Section VI measures.

5) Batch-STARK and recursion: A multi-table *batch*-STARK proves many AIRs of different heights under one shared commitment, with cross-table buses between them; Plonky3 [12] provides this together with a recursion crate whose verifier circuit re-verifies a batch-STARK proof whose Merkle hashes are Poseidon2 [13]. Recursion is what converts many proofs into one: the leaf proofs become witness data of an outer proof whose public inputs are the statements of the leaves.

B. Commitments and interactive oracle proofs

Transparent, plausibly post-quantum systems commit to polynomials by hashing a Reed–Solomon (or other linear-code) encoding into a Merkle tree and proving proximity to the

TABLE XII
SEGMENTATION OF THE 59-TABLE CUBIC RELATION INTO s SEGMENT SETS WITH A JOINT LOGUP TRANSCRIPT. ALL POINTS ARE COMPLETE TWO-PASS RUNS, EVERY SEGMENT VERIFIED.

s	peak RSS	total proof	time vs. s=1
1	25.80 GiB	5.55 MB	1.00
2	16.62 GiB	6.31 MB	2.25 (same-session)
4	8.51 GiB	7.73 MB	contended, not quoted
8	**5.45 GiB**	11.17 MB	contended, not quoted
16 (balanced cuts)	**4.33 GiB**	17.84 MB	2.0–3.03 (CPU bracket)

Total trace cells are identical at every s; the cost is the two-pass protocol. 4.33 GiB is the table-granularity floor (one 33 M-cell table).

TABLE XIII
COST OF VERIFYING ONE LEAF PROOF IN-CIRCUIT (EXACT GEOMETRY: 62 TABLES, MAX LOG HEIGHT 20, MMCS ARITY 4, FRI FOLDING ARITY 2, 20 ROUNDS, $q = 195$).

Per query	leaf hashes	compressions
preprocessed openings	47	8
main-trace openings	110	8
permutation openings	92	8
quotient openings	49	8
commit-phase rounds	20	56
per query	318	88 (406 total)
H_R at $q = 195$		79,170 permutations
c_H (cells per permutation)		320 (316 + 4 MMCS)
C_{Merkle}, unpadded / padded		25.3 M / 41.9 M cells
share of leaf main trace (4.25×10^8)		5.96% / 9.87%
non-Merkle (ALU, $c_{\text{ALU}} = 12$)		4.6–12.6 M cells
whole verifier vs leaf		11.4–15.8%
in-circuit frontier walk, ceiling		1.122× Merkle, 1.07–1.10× total

code. FRI [14] folds a univariate polynomial by halving $\log$ many times and spot-checks q positions, each check opening one Merkle path per folding round; WHIR [15], BaseFold [16] and Ligerito [11] give multilinear commitments with different opening trade-offs; Brakedown [17] is linear-time but has $\sqrt{n}$-size proofs. Every such system pays for two things the workload cannot avoid: encoding the committed columns (an NTT over the trace at the blowup rate) and hashing the encoded rows into the tree. On our chain the commitment phases are 79% of peak memory (Table X, Fig. 4) and, in the counted model of the exact-convolution chain, 99% of prover work (84% NTT, 15% hashing). Trace *width* is therefore what governs prover cost, which is why every optimisation in this paper is about committed cells.

C. Security parameters

A proof system's soundness is stated against a proximity-gap theorem for the code. The strongest (capacity-regime) conjectures for Reed–Solomon codes over large fields were disproved in late 2025 [18]; the Johnson-bound and unique-decoding analyses stand. This is not academic. An early configuration of the present system was assessed at 116 bits by capacity arithmetic (100 queries × blowup +16 grinding bits); the Plonky3 estimator over the real AIRs under the proven bound gives 58.8 bits for the same configuration. The estimator takes the actual constraint set (341 constraints through the lookup gadget, 62 AIRs, log trace height 20) and reports the minimum over every term: the low-degree test, the batch combination of the AIRs' constraints, the out-of-domain sampling and the commitment collision. A degree-2 challenge extension of Goldilocks cannot exceed 100.1 bits at any query count, blowup or grinding, because the batch-combination term saturates at the width of the challenge field. Moving the challenge field to the cubic trinomial extension $X^3 - X - 1$ (192 bits) gives 113.4 proven bits at 195 queries and moves the binding term to the hash (Table XVII). We report proven bits everywhere and treat conjectured figures as diagnostics. Orion's practical soundness failures [19] and the Fiat–Shamir attacks on GKR compositions [20] are the same lesson from other directions: quote a system with its version and its bound.

D. Provenance and roots of trust

The acquisition must enter the statement by a collision-resistant hash. The obvious cheap alternative, an algebraic fingerprint $F = \sum_i r^i D_i$ with a public evaluation point r, costs nothing in a GKR circuit (constant multiplication) and is tempting. It is broken: with r known before the witness is fixed, the map $D \mapsto F$ is linear with a large kernel, and an all-integer LLL [21], [22] over $\mathbb{F}_p^3$ finds int16-valued perturbations that leave F unchanged from 14 words upward, with maximum perturbation ± 21 at 64 words in 0.4 s, below ADC noise. Schwartz–Zippel does not apply when the point precedes the witness. Signing F does not make it collision-resistant. The replacement is Poseidon2 inside the proof at 1.75% of the chain; SHA-256 in circuit would cost 3.8× the chain (Table IV).

The capture component signs a record (acquisition geometry, word count, the four digest limbs) with a key the ground has pinned. TPM 2.0 [23] and DICE [24] supply the key protection, measured state and layered identity such a component needs, and RATS [4] the vocabulary for appraising its evidence; none of them authenticates sensor bytes by itself, and the architecture must route the sensor interface and DMA through the trusted boundary. Ordered Merkle trees [25] are the standard tool where subsets of the acquisition must later be disclosed. The ground verifier must check the record against its own pinned key before anything else, and the prover must have no fallback that recomputes the digest.

E. Families surveyed

Table III gives the decision-relevant verdicts for every family considered; the full survey with evidence labels is in the accompanying technical report. Three findings shape the rest of the paper. Direct FFT and convolution sumchecks (zkCNN [26], 33× over a gate-level GKR baseline at 2^{18} points) are the strongest evidence that structured proofs beat gate expansion, and they require a multilinear commitment. Systems over 31-bit fields (Circle-STARK, Hypercube) cannot host the chain's 59–62-bit intermediates. And a trusted native chain (secure DMA, authenticated bitstream, protected output accumulator, policy-bound signature) removes almost all proving work at the price of a larger trusted computing base, which makes it the practical answer wherever a customer accepts that base.

TABLE XIV
RECURSIVE AGGREGATION OF m REAL SEGMENT PROOFS (OF THE s=8 PROVENANCE RELATION) INTO ONE OUTER STARK. CUBIC FRI ARITHMETIC, D=1 WIDTH-16 POSEIDON2 HASH TABLE, q=195. EVERY ROW PROVES AND VERIFIES EXTERNALLY AT 113.4 PROVEN BITS; THE m=8 ROW ENFORCES THE FULL-SET CLOSURE $\sum_i \tau_i = 0$ IN-CIRCUIT.

m	inner AIRs	T_{build}	T_{prove}	T_{verify}	M_{prove}	$\lvert\pi_{\text{agg}}\rvert$	vs. raw	vs. m separate
1	7	0.34 s	12.99 s	41 ms	5.23 GiB	2,251,836 B	1.074×	–
2	17	11.8 s	26.8 s	41 ms	8.71 GiB	2,405,144 B	1.147×	1.87× smaller
4	32	25.8 s	51.2 s	43 ms	16.53 GiB	2,533,553 B	1.208×	3.56× smaller
8	62	41.4 s	101.2 s	45 ms	**30.32 GiB**	**2,722,924 B**	**1.298**×	6.62× smaller

Cost model: $H_R = 79{,}170$ Poseidon2 permutations per leaf verification, $c_H = 320$ cells, verifier circuit 11–16% of the leaf trace. Outer memory tracks padded cells at 107–133 B/cell; the outer ALU table is 72% of them at m=8. Proof size steps with the ALU log-height (18, 19, 20, 21); differences below ∼27 kB are varint noise.

TABLE XV
LEAF PROVER BACKEND ON THE s=8 PROVENANCE RELATION (BRACKETED $K_a \to P2 \to K_b$). RECURSION REQUIRES THE POSEIDON2 COMMITMENT.

Backend	T_{prove}	wall	peak RSS	8 leaf proofs
Keccak (a)	94.5 s	116.2 s	5.74 GiB	12.46 MB
Poseidon2	223.5 s	273.6 s	5.59 GiB	14.05 MB
Keccak (b)	94.6 s	–	–	–
γ_{P2}	**2.363**	2.353	0.97	1.128

Phase split, segment 2: quotient 0.99×, FRI 3.96×, remainder 3.19× – the tax is a pure hash/commitment kernel.

V. SAR ACQUISITION AND THE SHIP-DETECTION CHAIN

This section states exactly what is proved: what the raw data is, what the certified algorithm A_θ does to it, and in which arithmetic. Everything below is transcribed from the fixed-point reference implementation that both circuits mirror bit for bit; the pseudocode is the specification, not a sketch of it.

A. What the raw acquisition is

A stripmap SAR transmits a linear-frequency-modulated pulse (a chirp) at the pulse repetition frequency and records the complex baseband echo of each pulse at the ADC sampling rate. The acquisition is therefore a two-dimensional array

$$X \in \mathbb{Z}^{N_a \times N_r \times 2}, \qquad X[k,n] = (I,Q) \in [-2^{15}, 2^{15})^2,$$

indexed by pulse ("azimuth", slow time $\eta = k/\mathrm{PRF}$) and range sample ("fast time", $\tau = 2R_{\text{near}}/c + n/f_s$). For the reference scene $N_a = 512$, $N_r = 1024$, so the raw acquisition is $512 \times 1024 \times 2 \times 2$ bytes $= 2{,}097{,}152$ bytes. This int16 array in $(N_a, N_r, 2)$ layout is the *authenticated object*: it is what the capture component hashes and signs, what the prover hashes in-circuit, and the only form the canonicalisation code accepts (non-integral values are refused, never rounded, because rounding would authenticate a byte string the ADC never produced).

The simulator (Table V) produces X from a pinned forward model. A point scatterer i at closest slant range $R_{0,i}$, crossing the beam centre at slow time $\eta_{c,i}$ with relative speed v_i, contributes to pulse k and sample n

$$\begin{aligned} R_i(\eta) &= \sqrt{R_{0,i}^2 + v_i^2(\eta - \eta_{c,i})^2}, \quad \tau_{d,i}(\eta) = 2R_i(\eta)/c, \\ X_i[k,n] &= a_i\, w_a(\eta - \eta_{c,i})\, e^{-j4\pi R_i(\eta)/\lambda} \\ &\quad \cdot\, e^{j\pi K_r(\tau - \tau_{d,i})^2}\, \mathrm{rect}\!\left(\frac{\tau - \tau_{d,i}}{T_p}\right), \end{aligned} \tag{6}$$

where a_i is the calibrated amplitude from the vessel's radar cross section, w_a the two-way antenna pattern that limits the synthetic aperture to $|\eta - \eta_c| \le R_0 \tan\theta_{\text{null}}/v$ (414 pulses for the 24 m antenna), K_r the chirp rate (50 MHz over 5 μs) and T_p the pulse length (375 samples at 75 MHz). Vessels are lattices of such scatterers with a superstructure; sea clutter is circular Gaussian, white in the raw domain (the speckle texture of a SAR image is produced by the focusing, not baked in); thermal noise is 12 dB below clutter. The sum is scaled by the converter gain and landed on the int16 lattice by round-half-to-even with a saturating clip. Every random draw comes from a seeded stream, so the acquisition is a deterministic function of (scene, seed).

The antenna is 24 m, deliberately long: the synthetic aperture is then 414 pulses and the whole aperture fits a 512-pulse block, at the price of 12 m azimuth resolution. That choice is geometry, not a shortcut; a Sentinel-1-like 12.3 m antenna needs a 1,577-pulse aperture and was verified in a 2048-pulse block. Point targets focus to the exact pixel; the range impulse response is 3.91 m with a Hamming sidelobe of -41.3 dB against -42.8 dB theoretical; a moving target displaces in azimuth by $v_r R_0/V$ to within a pixel; range-cell migration is 0.48 samples, small enough that a two-tap interpolator suffices and large enough that a one-LSB error in an interpolation weight moves 28 SLC cells.

What raw data looks like is the top-left panel of Fig. 2: no vessel is visible. Each scatterer is smeared over $T_p f_s = 375$ samples in range by the chirp and over 414 pulses in azimuth by the aperture, with the hyperbolic phase history above; the energy of a vessel is spread over roughly 1.5×10^5 cells at a level below the clutter. Focusing is the operation that collapses each of those histories to a point, and it is what an on-board detector must compute before anything can be detected.

For the provenance digest the acquisition is read as a stream of $2N_aN_r = 1{,}048{,}576$ 16-bit words (I then Q, raster order), packed three words per Goldilocks element (48 bits) and absorbed after a six-element header $(\text{SAR_RAW_V1}, N_{\text{words}}, N_a, N_r, \text{format}, 16)$ by a rate-12

TABLE XVI
EVOLUTION OF THE BATCH-STARK RELATION ON THE 512×1024 SCENE. ALL CONFIGURATIONS VERIFY AND REPRODUCE THE FIXED-POINT REFERENCE BIT-EXACTLY (241 DETECTIONS / 18 COMPONENTS / 13 CANDIDATES). TIMES MARKED (P) WERE MEASURED ON A LOADED HOST AND ARE ONLY COMPARABLE WITHIN A ROW GROUP.

Date	Configuration	tables	cells/px	peak RSS	T_{prove}	T_{verify}	$\lvert\pi\rvert$	proven bits
08-21	frozen, bit-column range checks, Keccak, D=2, q=100	58	2,115	38.9 GiB	∼100 s	16 ms	3.57 MB	58.8*
08-22	LogUp range table, Keccak, D=2, q=100	59	784	22.95 GiB	70.8 s	11.8 ms	2.46 MB	58.8
08-24	cubic extension X^3-X-1, q=200	59	784	25.80 GiB	100.9 s (P)	35 ms	5.55 MB	**113.4**
08-26	+ Poseidon2 provenance (h_{raw} public), q=195	62	811	n/r	87–89 s (P)	n/r	n/r	113.3
09-07	demonstrator, signed capture record, q=195	62	811	n/r	192.8 s‡	92 ms	n/r	113.3

*recomputed under the proven (Johnson) bound; an earlier capacity-conjecture estimate gave 116 bits. n/r: not recorded. ‡includes recursion instrumentation (∼200 s in the 08-27 battery).

TABLE XVII
PROVEN SOUNDNESS OF THE GOLDILOCKS BATCH-STARK AS A FUNCTION OF CHALLENGE-FIELD DEGREE AND FRI QUERY COUNT (LOG BLOWUP 1, 16 POW BITS), COMPUTED BY THE PLONKY3 ESTIMATOR OVER THE REAL AIRS.

queries	D=2 (128-bit)	D=3 (192-bit)	binding term at D=3
100	58.8	65.9	low-degree test
170	–	100.9	low-degree test
195	–	**113.4**	batch combination
200	99.0	113.4	batch combination
300	100.1 (cap)	128.0	commitment collision

Cost of D=2 → 3: 1.20× prover time, 1.12× peak RSS, 2.25× proof bytes at q=200, output byte-identical. Prove time is flat in q.

overwrite sponge: 349,532 elements, 29,128 Poseidon2 permutations, digest h_{raw} = the first four lanes of the final state.

B. Fixed-point conventions

A value is a pair (raw integer, format), with format $Q(w.f)$ meaning a w-bit signed word whose least significant bit is 2^{-f}; f may be negative. There is one rounding rule, and every stage uses it:

$$\mathrm{rs}(x,k) = \left(x + 2^{k-1}\right) \gg k \qquad (k>0), \tag{7}$$

an arithmetic shift with round-half-up, exact for negative x. A change of format is $\mathrm{rs}(x, f_{\text{src}} - f_{\text{dst}})$. Public real constants (θ's ADC scale and intensity gain) enter as dyadic rationals $\mathrm{num}/2^{\text{shift}}$ with a stated mantissa, never as floats. Every product is bounded by 2^{62} so that it fits Goldilocks with a guard bit; the bounds are derived analytically from $|\mathrm{DFT}_N(y)| \le N \max|y|$ and the constants' magnitudes, not from data, so the circuit's shape is a function of θ and the metadata only. The formats for the reference scene are in Table VI: samples are 43 bits wide throughout, the fractional point moves from 20 (conditioned) through 9 (range spectrum), 5 (range compressed) to -4 (SLC); the references H_r and H_a are 19-bit with 12 and 17 fractional bits; the pre-square quantiser is 31 bits.

C. The stages, as pseudocode

The certified relation is the range-Doppler algorithm [27], [28] followed by a detector (Fig. 2). Algorithms 1 to 9 are the stages in order; the constants in them are the reference scene's (Table VI).

Algorithm 1 Conditioning: $X' = \mathrm{rs}(((X \ll 16) - dc) \cdot n_{\text{adc}},\ 16)$

1: $n \leftarrow N_a N_r$; $dc_I \leftarrow \lfloor (2 \cdot 2^{16} \sum X_I + n)/(2n) \rfloor$, same for dc_Q ▷ round-half-up mean, 16 frac bits
2: **for all** (k,n) **do**
3: $c_I \leftarrow (X_I[k,n] \ll 16) - dc_I$; $c_Q \leftarrow (X_Q[k,n] \ll 16) - dc_Q$
4: $X'_I[k,n] \leftarrow \mathrm{rs}(c_I \cdot 12{,}582{,}912,\ 16)$; $X'_Q[k,n] \leftarrow \mathrm{rs}(c_Q \cdot 12{,}582{,}912,\ 16)$ ▷ $Q(43.20)$
5: **end for**

1) Conditioning: The only data-derived number in the chain is the DC estimate, and the only division is its mean; in a circuit $N_a N_r$ is a constant of θ so this is a multiply by a reciprocal. Each sample is then centred, scaled by the ADC gain $12{,}582{,}912/2^{20}$ and rounded *once*, because two roundings would put a half-LSB bias into every sample before the matched filter sees it (Algorithm 1).

2) The transform: All four transforms are the same radix-2 decimation-in-time FFT with an *unconditional* one-bit down-shift per stage (Algorithm 2). Block floating point would make the shift a function of the data, i.e. control flow inside a fixed arithmetic circuit; one bit per stage is a constant. After $L = \log_2 N$ stages the raw integers hold DFT/N; the $1/N$ is booked into the format (the forward transform lowers f by L; the inverse leaves it, because the reference's inverse FFT carries the $1/N$ itself), so the values track the float reference stage for stage. Because $|\mathrm{DFT}_M(y)| \le M \max|y|$, no intermediate exceeds the input magnitude and the word width never grows. Twiddles $W_N^k = e^{-2\pi jk/N}$, $k < N/2$, are rounded half-up into $Q(1.15)$ (17 bits, so that $\cos 0 = 1$ is representable); one table serves every stage by striding; the inverse transform conjugates it. Each butterfly performs six real requantizations, and these, not the multiplications, are the proof's cost.

3) Range compression: Each pulse is zero-padded to $N_{\text{fft}} = 2048 \ge N_r + T_p f_s - 1$ (so the circular correlation cannot alias far range onto near range), transformed, multiplied by the reference H_r, inverse-transformed and cropped to N_r (Algorithm 3). The reference is

$$H_r[m] = \overline{\mathrm{DFT}_{N_{\text{fft}}}[h]}[m] \cdot W_r[m],$$
$$h[n] = e^{j\pi K_r \tau_n^2}\,\mathrm{rect}(\tau_n/T_p), \qquad \tau_n = (n - c_0)/f_s, \tag{8}$$

with the replica circularly centred on sample zero ($c_0 = \lfloor T_p f_s/2 \rfloor$) so that compressed sample n corresponds to slant range $R_{\text{near}} + nc/(2f_s)$ with no half-sample bias, and W_r

TABLE XVIII
ADVERSARIAL VALIDATION. EACH BATTERY IS READ HONEST-BASELINE FIRST; "(VERIFY, EXACT)" IS (PROOF ACCEPTED, OUTPUT MATCHES THE REFERENCE).

Battery	Case	Models	Result
Chain (08-26)	0 honest	–	(T, T)
	1 substitute the raw acquisition	malware replaces D	(F, F); verified before provenance
	2 perturb RCMC$\rightarrow H_a$ seam	corrupt intermediate	(F, T)
	3 hide one detection	suppress a vessel	(F, T)
	4 alter one published pixel	forge imagery	(F, T)
	5 / 6 split / merge a component	change vessel count	(F, F)
	7 alter a chip pixel	forge delivered chip	(F, T)
	8 fake a group boundary	manipulate grouping	(F, F)
	9 drop a qualifying candidate	omit a detection	(F, F)
Provenance (08-25/26)	10 corrupt a hash-side raw receive	break dual witness	(F, T)
	11–14 packer sign, P2 column, lane swap, packer copy	forge the digest path	all (F, T)
	15–17 seam value, seam lanes, boundary index	break sharded sponge	all (F, T)
	18 honest witness, wrong public digest	wrong acquisition claimed	(F, T)
Segment sets (08-21)	A1 omit a segment	partial proof set	rejected
	A2 terminals $+1/-1$, sum preserved	forge cross-segment closure	both segments fail
	A3 splice a segment from another run	mix acquisitions	rejected
	A4 re-check against another cut list	wrong statement	rejected
Recursive verifier (08-27)	`digest`	wrong segment digest	rejected, witness 4371
	`batch`	wrong batch digest	rejected, witness 0 (desync)
	`layout`	wrong lookup layout	rejected, witness 1,728,946 (lookup)
	`assoc`	another segment's legitimate framing	rejected, witness 4371
Demonstrator (09-07)	forgery, key swap, sig/digest/geometry tamper	capture-record attacks	no proof attempted (5/5)
	valid record of another acquisition	provenance separation	verify F, rejected at carrier AIR
	missing record, trusted key omitted	fail-closed	no proof attempted (2/2)

TABLE XIX
MEASURED POINTS ON THE ALTERNATIVE ROUTES.

Route	Workload	size	T_{prove}	peak RSS	$\lvert\pi\rvert$	note
Flock, Boolean R1CS	SHA-256-bound chain to CFAR, 8×8, 8 batched	m=23, 705 k AND	4.49 s	1.5 GB	283 kB	1.83 GB instance
Flock, decomposed	ingest/focus/detect, 32×32	3 proofs, 18.3 M AND	203 s	46.27 GB	1.02 MB	largest completed
Flock, tile	2 pulses $\times$ 512 range	m=29	32.6 s	79.91 GB	–	$k_{\log}$=26
Flock, synthetic ceiling	registered maximum	m=35	260.7 s	123.3 GB	–	68.7 GB JSON
Expander GKR	128 range lines: NTT conv. $+\lvert\cdot\rvert^2+$ az. sum	8 proofs	1.10 s	443 MB	8.45 MB	0 mul gates; proof 4$\times$ raw
Expander GKR	magnitude, 2^{24} samples	2 mul/sample	2.89 s	19.2 GB	268 MB	Raw PCS, 16 B/sample
Plonky3 uni-STARK	operational butterfly, sound	per butterfly	2.5 μs	–	–	12$\times$ Expander
WHIR (PCS only)	2^{29} cells, cubic, fold 8, 100-bit	–	7.72 s	20.68 GiB	847 KiB	parity with FRI (22.95 GiB)

TABLE XX
BYTES TO THE GROUND FOR THE 512×1024 SCENE. RAW IS THE AUTHENTICATED U16 I/Q STREAM; THE I64 FILE LAYOUT (8.39 MB) IS STORAGE, NOT WIRE.

Object	bytes	vs. raw
raw acquisition, u16 I/Q	2,097,152	1.000
detections + chips (products)	125,829	0.060
single proof, Keccak, D=2 (58.8 b)	2,461,841	1.174
single proof, cubic, q=200, 59 tables	5,550,000	2.65
8 segmented leaf proofs, Poseidon2	14,050,000	6.70
8 separate recursive proofs	18,014,688	8.59
one recursive proof of one segment	2,251,836	1.074
recursive aggregate m=8, q=195 (113.4 b)	2,722,924	1.298
recursive aggregate m=8, $q_{\min}$=169 (100.4 b)	2,397,028	1.143

No measured configuration is below raw at ≥ 100 proven bits: $\lvert\pi\rvert = 278{,}704 + 12{,}534.5\,q$ reaches raw only at q=145, 24 queries below the 100-bit minimum; with products the q=169 aggregate is 1.203$\times$ raw. The wire format is at its floor (canonical 8-byte field elements).

TABLE XXI
END-TO-END DEMONSTRATOR (`SAR_DEMO`), 2026-09-07: PINNED CAPTURE KEY, SIGNED ACQUISITION RECORD, NO PROVER FALLBACK.

Quantity	value
regression cases (honest + hostile)	12 / 12
T_{prove} (incl. trace generation)	192.8 s
T_{verify}	92 ms
proven security, leaf relation	113.3 bits
detections / candidates / components	241 / 13 / 18
chain constants SHA-256	`0fed4d99...`
h_{raw} equals signed record	yes (4 limbs)
candidate leaf RAM, s=8, Poseidon2	5.59 GiB
leaf prove, s=8, Poseidon2	223.5 s
flat m=8 aggregate at outer $q_{\min}$=169	2,397,028 B (1.143$\times$ raw)

a Hamming taper $0.54 - 0.46\cos(2\pi i/(M-1))$ over the $2k_{\max}+1$ bins of the 50 MHz band and zero outside it. H_r is quantised half-up into $Q(19.12)$ and is a preprocessed constant of the AIR; its zero bins cost nothing.

4) Azimuth transform and range-cell migration correction: Each range bin is transformed along azimuth (length $N_a = 512$, no window, no shift), giving the range-Doppler domain with bin k at $f_a = ((k + N_a/2) \bmod N_a - N_a/2)\,\mathrm{PRF}/N_a$ unwrapped about the Doppler centroid f_{dc} derived from the signed attitude. A target's range then curves with azimuth

TABLE XXII
WHAT TO BUILD DEPENDS ON THE CUSTOMER'S REQUIRED STATEMENT, NOT ON THE PROOF BACKEND .

Customer requirement	recommendation	immediate limitation
verify selected, requested SAR regions; trust only capture on board	authenticated raw support + selective ground replay of on-board chips	saves bandwidth only if the complete dependency set is small; no whole-scene completeness guarantee
verify requested raw bursts	authenticated extraction (Merkle inclusion against the capture root)	proves selection and copy, not focusing or detection
compact autonomous detections with conditional confidence	protected native sensor-to-processor-to-signature chain	trusted hardware, firmware, DMA and software replace the proof; a TPM signature alone is insufficient
complete autonomous detection with capture-only trust and arbitrary malicious software	**no practical winner established; do not promise one**	full proofs, structured proofs and streaming provers remain research at this scale

Algorithm 2 Fixed-point radix-2 DIT transform of length $N = 2^L$ (per line; batch over the other axis)

$x \leftarrow x[\mathrm{bitrev}_N(\cdot)]$ ▷ bit-reversal permutation, folded into addresses in the AIR
$\sigma \leftarrow -1$ if inverse else $+1$
for $s = 1 \dots L$ **do** ▷ $m = 2^s$, half $= 2^{s-1}$, stride $= N/m$
  for each block of m elements, each $j < m/2$ **do**
    $(w_r, w_i) \leftarrow (\mathrm{TW}_r[j \cdot N/m],\ \sigma \cdot \mathrm{TW}_i[j \cdot N/m])$ ▷ $Q(1.15)$ constants
    $a \leftarrow x[j]; \quad b \leftarrow x[j + m/2]$
    $t_r \leftarrow \mathrm{rs}(b_r w_r - b_i w_i,\ 15); \quad t_i \leftarrow \mathrm{rs}(b_r w_i + b_i w_r,\ 15)$
    $x[j] \leftarrow \big(\mathrm{rs}(a_r + t_r, 1),\ \mathrm{rs}(a_i + t_i, 1)\big)$
    $x[j + m/2] \leftarrow \big(\mathrm{rs}(a_r - t_r, 1),\ \mathrm{rs}(a_i - t_i, 1)\big)$
  end for
end for
return x ▷ forward: $f \leftarrow f - L$; inverse: f unchanged

Algorithm 3 Range compression, per pulse k

$p \leftarrow (X'[k, 0{:}N_r],\ 0^{N_{\mathrm{fft}} - N_r})$ ▷ literal zeros in the sample format
$S \leftarrow \mathrm{FFT}_{2048}(p)$ ▷ Alg. 2; $Q(43.20) \to Q(43.9)$
for all $m < N_{\mathrm{fft}}$ **do**
  $F_r[m] \leftarrow \mathrm{rs}(S_r[m]H_{r,r}[m] - S_i[m]H_{r,i}[m],\ 16); \quad F_i[m] \leftarrow \mathrm{rs}(S_r[m]H_{r,i}[m] + S_i[m]H_{r,r}[m],\ 16)$
end for
$y \leftarrow \mathrm{IFFT}_{2048}(F); \quad \mathrm{rc}[k, \cdot] \leftarrow y[0{:}N_r]$ ▷ $Q(43.5)$

Algorithm 4 Azimuth FFT and RCMC

for all range bins n **do** $\mathrm{spec}[\cdot, n] \leftarrow \mathrm{FFT}_{512}(\mathrm{rc}[\cdot, n])$ ▷ $Q(43.5) \to Q(43.{-}4)$
end for
for all (k, n) **do** ▷ i_0, i_1, w_0, w_1 are preprocessed constants
  $\mathrm{mig}[k, n] \leftarrow \mathrm{rs}\big(w_0[k, n]\,\mathrm{spec}[k, i_0[k, n]] + w_1[k, n]\,\mathrm{spec}[k, i_1[k, n]],\ 15\big)$ ▷ both planes
end for

Algorithm 5 Azimuth compression and the SLC

for all (k, n) **do**
  $F_r \leftarrow \mathrm{rs}(\mathrm{mig}_r H_{a,r} - \mathrm{mig}_i H_{a,i},\ 17); \quad F_i \leftarrow \mathrm{rs}(\mathrm{mig}_r H_{a,i} + \mathrm{mig}_i H_{a,r},\ 17)$ ▷ H_a stored conjugated
end for
for all range bins n **do** $\mathrm{slc}[\cdot, n] \leftarrow \mathrm{IFFT}_{512}(F[\cdot, n])$ ▷ $Q(43.{-}4)$, the focused image
end for

frequency by

$$\Delta R(f_a, R) = \frac{\lambda^2 R f_a^2}{8V^2}, \quad \delta[k, n] = \frac{2f_s}{c}\,\Delta R(f_a[k], R(n)), \tag{9}$$

a rank-one surface in (f_a^2, R) that peaks at 0.48 samples on this geometry. It is straightened by a two-tap linear interpolation whose source indices and weights are constants of θ (Algorithm 4): $i_0 = \lfloor n + \delta \rfloor$, $i_1 = i_0 + 1$, $w_1 = \lfloor (n + \delta - i_0) 2^{15} + \frac{1}{2} \rfloor$, $w_0 = 2^{15} - w_1$; a tap that would read beyond the swath gets weight 0 and index 0, so the gather is always in bounds and there is no data-dependent branch. The fixed chain reads exactly the samples the float chain reads; only the weight is quantised.

5) Azimuth compression: The azimuth reference is the windowed matched filter

$$H_a[k, n] = W_a[k]\, e^{\,j\pi f_a[k]^2 / K_a(R(n))}, \qquad K_a(R) = \frac{2V^2}{\lambda R}, \tag{10}$$

applied conjugated (the literal, unconjugated reading defocuses by $2\times$; the conjugation is baked into the stored constant), with W_a the Hamming taper over the processed Doppler band centred on f_{dc}. It is range-dependent, so it is an $N_a \times N_r$ table of $Q(19.17)$ constants. One complex multiply with a 17-bit shift per pixel, then the inverse azimuth transform yields the single-look complex image (Algorithm 5).

6) Intensity: Squaring a 43-bit SLC needs 86 bits, so the SLC is first requantised into a 31-bit *saturating* format whose full scale is the magnitude at which the output code saturates anyway ($\sqrt{\max_{\mathrm{code}}/g}$); any sample the clip touches would have produced the maximum code regardless, so no output bit changes. The power $I^2 + Q^2$ (never $|z|^2$ via a square root) is then scaled by the calibration gain as a 30-bit dyadic constant with a 57-bit shift; the 91-bit product is formed exactly (two 31-bit limbs in the reference; the natural wide multiply in the circuit) and clipped to 32 bits (Algorithm 6).

7) Detection: integral image and CA-CFAR: The detector is cell-averaging CFAR made division-free. An integral image J of I is built by two running sums (range, then azimuth); the sum over any rectangle is then four corner reads. For each cell the background B is the sum over the 25×25 outer window minus the 9×9 guard window ($N_T = 625 - 81 = 544$ training cells), floored at 1, and the decision compares the cell to $\alpha = 130/10$ times the mean background by cross-multiplication (Algorithm 7). Cells whose outer window overhangs the image are invalid and never detected ("skip" border). Q is at most 2^{58} in magnitude; its sign is what the circuit proves, by a witnessed 40-bit decomposition. $\alpha = 13$ gives a false-alarm rate near 2×10^{-6} on Rayleigh clutter.

8) Connected components, grouping, candidates, chips: Detected cells are grouped into 8-connected components labelled in raster order of their first pixel (Algorithm 8); the reference is a two-pass union-find, and the circuit does not re-run it but verifies a prover-supplied spanning forest: each

Algorithm 6 Intensity: $I = \min(\mathrm{rs}(g \cdot (\hat{I}^2 + \hat{Q}^2)),\ 2^{32} - 1)$

for all (k, n) **do**
  $\hat{I} \leftarrow \mathrm{clip}_{31}(\mathrm{rs}(\mathrm{slc}_r, 0))$; $\hat{Q} \leftarrow \mathrm{clip}_{31}(\mathrm{rs}(\mathrm{slc}_i, 0))$ ▷ saturating $Q(31.-4)$
  $P \leftarrow \hat{I}^2 + \hat{Q}^2$ ▷ $< 2^{62}$
  $c \leftarrow (P \cdot 885{,}443{,}716 + 2^{56}) \gg 57$ ▷ exact 91-bit product
  $I[k, n] \leftarrow \min(c,\ 2^{32} - 1)$
end for

Algorithm 7 Integral image and division-free CA-CFAR

$J[k, n] \leftarrow \sum_{k' \le k, n' \le n} I[k', n']$ ▷ two prefix passes; J has a zero border row and column
for all (k, n) with $12 \le k < N_a - 12$, $12 \le n < N_r - 12$ **do** ▷ valid cells
  $S_{\mathrm{out}} \leftarrow \mathrm{box}(J, k \pm 12, n \pm 12)$; $S_{\mathrm{g}} \leftarrow \mathrm{box}(J, k \pm 4, n \pm 4)$ ▷ 4 corner reads each
  $B \leftarrow \max(S_{\mathrm{out}} - S_{\mathrm{g}},\ 1)$; $N_T \leftarrow 544$
  $Q[k, n] \leftarrow 10 \cdot N_T \cdot I[k, n] - 130 \cdot B$
  $D[k, n] \leftarrow [\, Q[k, n] > 0 \,]$
end for
$Q, D \leftarrow 0$ on invalid cells

Algorithm 8 Connected components (reference; the circuit verifies a certificate of the result)

for all detected (k, n) in raster order **do** ▷ pass 1
  $\mathcal{N} \leftarrow$ labels of the causal 8-neighbours $(k{-}1, n{\pm}1), (k{-}1, n), (k, n{-}1)$ that are detected
  if $\mathcal{N} = \emptyset$ **then** new label
  else $\ell \leftarrow \min \mathcal{N}$; union every other member of $\mathcal{N}$ with ℓ
  end if
end for
for all detected (k, n) in raster order **do** ▷ pass 2
  $\mathrm{lab}[k, n] \leftarrow$ final number of $\mathrm{find}(\ell)$, numbers assigned in order of first appearance
end for

Algorithm 9 Grouping, candidates and chips

for all labels ℓ **do**
  $\mathrm{area}_\ell \leftarrow |\{(k, n) : \mathrm{lab} = \ell\}|$; $(\mathrm{peak}_\ell, k^*_\ell, n^*_\ell) \leftarrow$ first raster maximum of Q over the component
  $\mathrm{pass}_\ell \leftarrow [\, 2 \le \mathrm{area}_\ell \le 4096 \,] \wedge [\, \mathrm{peak}_\ell \ge 0 \,]$
end for
$C \leftarrow$ passing labels sorted by $(-\mathrm{peak}, k^*, n^*, \ell)$, truncated to 256
for all $\ell \in C$ **do** $\mathrm{chip}_\ell \leftarrow I[k^*_\ell - 32 : k^*_\ell + 32,\ n^*_\ell - 32 : n^*_\ell + 32]$ ▷ one bus receive per pixel in the AIR
end for
return $Y = (I, Q, D, \{(\ell, k^*, n^*, \mathrm{peak}, \mathrm{area})\}_{\ell \in C}, \{\mathrm{chip}_\ell\})$

detected cell carries a label, a one-hot parent among its eight neighbours and a distance to the root, and the constraints are that a root's label is its own raster index plus one, that a child's distance is its parent's plus one (so the forest is acyclic), and that adjacent detected cells carry equal labels (so components are not split). For each label the pixels are re-presented sorted by (label, raster index), which the circuit proves is a permutation by a bus, and the area and the peak CFAR score with its position are accumulated (first maximum in raster order on ties). A group is a candidate if $2 \le \mathrm{area} \le 4096$ and its peak score is at least 0; candidates are ordered by peak score descending with raster tie-break and truncated to 256 (the truncation never fired on the reference scene and is not modelled in-circuit). For each candidate the 64×64 window of I centred on its peak is the published chip, $\mathrm{chip}[32, 32] = I[\mathrm{peak}]$; the prover refuses candidates within half a chip of the border so that only the interior case is certified (Algorithm 9).

D. What is inside and outside the proof

Inside: everything in Algorithms 1–9 from the raw words to the chip pixels, with θ (ADC gain, twiddles, H_r, H_a, RCMC tables, CFAR constants, grouping thresholds) as preprocessed constants, plus the provenance digest of the raw words. Outside, by design: multilook (identity at 1×1), geolocation of the peak to latitude and longitude, vessel length and heading from the component's second moments, and AIS correlation, all of which the verifier recomputes from the certified product; and the physics of the simulator itself. Not certified because the prover refuses the case: intensity clipping (the reference scene has zero clipped samples), chips overhanging the border, and candidate truncation at 256.

Against the float reference the fixed chain reproduces every CFAR decision and every component; the worst per-stage SNR is above 60 dB. The prover asserts byte-equality with the reference at seven checkpoints (conditioned, range compressed, azimuth spectrum, SLC, codes, CFAR, candidates) and refuses to prove otherwise. The operation mix per pixel is field-independent and decides everything downstream: about 148 multiplications, all by public constants; about 210 additions; about 200 right-shifts (the roundings in Algorithms 2 to 6); 2 gathers; 1 division. A prime field gives the first two for free. *The cost of proving SAR is the cost of proving its roundings.*

E. A note on the prescreener study

Before the proof work, a separate study asked whether a proof-friendly range filter (the 1024-tap reference quantised to $\{1, i, -1, -i\}$) and a selective fine chain could replace full processing. Its results are in Table VII: the quantised filter is statistically indistinguishable from the reference except at land edges (the deficit is caused by taper removal, not phase quantisation), a hard per-scene budget scheduler is exact, and selective processing saves downlink and RAM but not compute, because the coarse pass range-compresses the whole scene a second time. Coefficient simplicity turned out to buy nothing in a prime-field STARK, because twiddles and filter taps are preprocessed constants; the study is recorded for completeness and does not bear on proof cost.

VI. Arithmetizing the DSP Chain

We built the same chain twice, under the two arithmetizations most often proposed for integer-heavy workloads, and a third partial encoding for the linear stages. Table VIII summarises the per-pixel cost.

A. Boolean circuit over $\mathbb{F}_2$ (Flock)

The first route arithmetizes the fixed-point chain as an AND/XOR circuit and proves it with Flock. Only AND gates are committed (plus inputs, outputs and linear relays); XOR is free. The gadget library (Table IX) reaches the known-optimal AND counts: an n-bit add costs $n - 1$, a 16×16 multiply 496 against a floor of 480, a round-shift by k

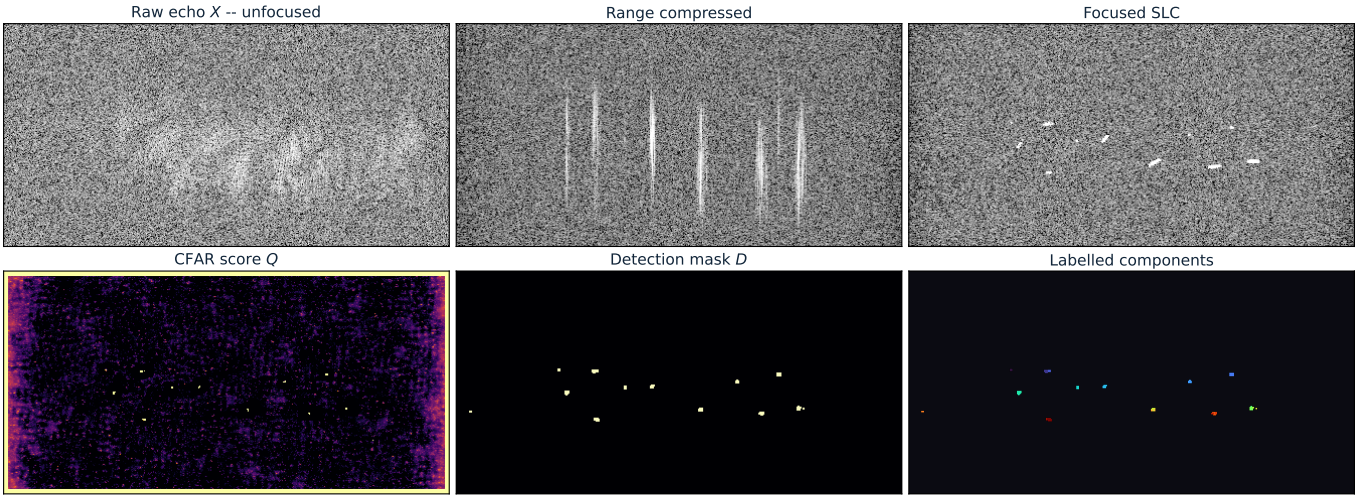


Fig. 2. Six of the certified intermediate products for the reference scene: raw echo (no vessel visible; each scatterer is spread over 375 range samples by the chirp and 414 pulses by the aperture), range compressed (the chirp collapsed, the azimuth histories still smeared), focused SLC, CFAR score Q, detection mask D, labelled components. Every stage between them is an AIR, and every arrow is a lookup bus.

costs $n - k$. Multiplication by a public constant uses non-adjacent-form recoding of the constant so that a 17-bit twiddle averages 5.3 nonzero digits instead of 8.1, and the rounding is folded into the final carry chain: a butterfly costs 1,137 ANDs against 1,842 with binary recoding and 5,764 with a general multiplier. The input is bound by SHA-256 on the same wires at 22,573 ANDs per 512-bit block, the standard Bristol figure. Connected components are verified from prover-supplied advice (labels, parents, depths) with five checks (a label iff detected; adjacent cells agree; parent strictly closer to the root; root ranks canonical; count bounded) at about 200 ANDs per pixel.

Two engineering findings from this route transfer to any Boolean prover. First, substituting XOR wires away as affine forms is free in committed bits and ruinous in matrix density: a 32-point range line had a mean XOR support of 1,454 and 790 million nonzeros; giving wires above a support threshold their own column ("linear relays", paid from per-block padding) cut density $592\times$. Second, an uncommitted intermediate state can be referenced across proofs by an affine *boundary seam*: its multilinear fingerprint at a random point is a linear functional of the witness, folded by a reverse walk over the XOR cone and discharged by the existing lincheck, at 0.71 ms of prover work and 2–3 kB of proof, avoiding both materialising and hashing the boundary. On the theory side, LogUp is unsound in characteristic 2 (an even multiplicity vanishes), so a lookup argument for this setting needs a different construction; a Frobenius-based multiplicative one was drafted and machine-checked in $\mathbb{F}_{2^8}$ but not completed.

The chain was built and verified element by element at 8×8 through 64×64 (79.7 M ANDs, 60 GB to build), and real proofs were produced at 8×8 (one bound statement, $m = 23$, 4.5 s, 283 kB, a 1.83 GB instance) and 32×32 (three proofs, 203 s, 46 GB peak). Measured on the committed-column axis, the route costs 498 columns per pixel and 16.8 proof bytes per pixel at a 4 GB prover budget, against about 2 bytes per pixel to downlink the detected image itself. A 43-bit SAR sample is 43 committed columns here and one committed cell in the next route. Two further measurements closed the route: prover residency tracks $2^{k_{\log}}$ at about 1.4 kB per base-block row (a 2-pulse ingest tile needed 80 GB), and end-to-end throughput was 7.6 ns per witness bit on sparse synthetic instances but 1,679 ns per bit on the dense SAR statement, not the 0.74 ns per bit of the Flock paper's hash benchmarks. Bit-exact binary-field arithmetization of this chain is roughly four orders of magnitude too expensive for on-board use. Flock remains a candidate co-prover for the hash-heavy part of a statement.

B. Multi-table AIR over Goldilocks (Plonky3 batch-STARK)

The second route is the one carried to completion. Each processing stage is an AIR; the 62 tables (one per FFT stage, one per filter multiply, and one each for conditioning, RCMC, intensity, two integral-image passes, CFAR, components, grouping, candidates, chips, plus the range table and the three provenance tables) are proved as one batch-STARK under one FRI commitment. The load-bearing design decision is that every intermediate value carries a *preprocessed* address, and every producer sends and every consumer receives $(\text{address}, \text{re}, \text{im})$ on a LogUp permutation bus. Addresses follow a static plan: range-domain level ℓ, pulse k, bin p maps to $(\ell \cdot n_a + k) \cdot n_{\text{fft}} + p$, azimuth-domain levels follow after the range block, and the intensity, integral-image, detection and component blocks after those, so that the bit-reversal permutation of each transform and the corner turn between range and azimuth are absorbed into preprocessed addresses at no cost. The verifier fixes the dataflow; the prover cannot choose it; wiring is not asserted but is the only way the proof closes. Seven named buses carry the chain, the component adjacency, the grouping, the candidates, the range checks, the provenance sponge and the shard seam. A butterfly AIR is about twenty lines. Rounding is proven as $v + 2^{k-1} = q2^k + r$ with r range-checked, first by k boolean columns (2,115 cells per pixel) and then by a shared 2^{15}-row lookup table with 15-bit limbs (784 cells

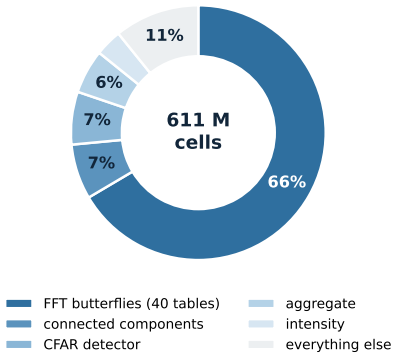


Fig. 3. Where the 611 M committed cells of the unsegmented chain go. The forty butterfly tables of the four transforms are two thirds of the proof; everything after focusing is under a fifth.

per pixel, 1,165 including the lookup helper columns). The cell budget is 44.4% right-shifts and range checks, 32.7% wiring, 13.2% data values carried between stages and 9.6% comparison, labelling and aggregation logic (Table X, Fig. 3). Against the 561 arithmetic operations per pixel this is 2.08 cells per operation; switching proof systems buys a constant factor from here, not an order of magnitude.

Two AIRs deserve description because they are where the detector's semantics live. The CFAR AIR receives eight corners of the integral image, forms the outer and guard box sums by inclusion-exclusion, pins $\max(B, 1)$ by $z \cdot B = 0$, $B \cdot B^{-1} = 1 - z$, and proves the sign of Q by a witnessed $u = d(Q-1) + (1-d)(-Q) \in [0, 2^{40})$. The component AIR receives, for each detected cell, a label, a one-hot parent among its eight neighbours and a distance to the root, and constrains that a root's label is its own index, that a child's distance is its parent's plus one, and that adjacent detected cells carry equal labels; the grouping AIR then re-presents the cells sorted by (label, raster index), proves the permutation by a bus, and accumulates area and peak with two-sided witnessed flags $u = fx + (1-f)(-x-1)$ for every prover-chosen boolean. The last form is the repair of a real defect: a flag that is only constrained to be boolean is a licence to choose the outcome.

Soundness of an encoding of this size is not established by inspection. Four defects were found by adversarial testing during construction, none by reading the constraints: a preprocessed group-end flag the prover could choose; a proof that was verified in-process and never emitted, so that the thin verifier initially accepted a tampered chip file; the half-constrained candidate-filter flags above, which let a qualifying candidate be dropped with the proof still verifying; and an AIR and its witness generator that had drifted together in a fork, so that the pair was self-consistent and wrong. The methodological lesson is general: *a defect that lives in a constraint and its witness generator is invisible while the two move together, and a test suite built from the same pair cannot see it.* The checklist that follows is four questions per table: could the prover have chosen this preprocessed value; which published file still verifies after a bit flip; which prover-chosen flags are free; does the fork still prove the same relation.

C. GKR for the exact linear stages (Expander)

The third encoding used a GKR prover [29] for the coarse, exact-integer part of the chain: range compression by an 8192-point NTT with public twiddles, magnitude, and azimuth summation. A linear map with public coefficients compiles to zero multiplication gates, so a real 4096-sample range line proves in 63 ms and a 128-pulse block in 1.1 s at 443 MB of flight-side RAM, constant in scene size (Table XIX). But the proof volume under the Raw commitment is $4\times$ the raw bytes, the two-backend split (GKR for the linear stages, a STARK for the roundings) has no shared commitment to bind the two proofs to one acquisition, and the input binding used the fingerprint later broken. The structural lesson survives: linear stages want a sumcheck, not gates.

D. The counted alternative: exact convolution

The rounding FFT is a design choice of the DSP chain, not of the proof system. An exact integer cyclic convolution with one requantization (implemented in the reference as `range_compress_exact_fixed`, 120 dB against 91 dB for the rounding transform) removes the six roundings per butterfly. Counted on the same word widths (Table XI), the chain then costs 40 cells per pixel and 0.81 proof bytes per pixel, a factor 10.9 below the as-built 436; 88% of the as-built cells are in the transforms. Two corrections to the original specification came out of the count. The chain has five requantization points (DC estimate, ADC gain, range, azimuth, intensity gain) plus three saturating clips, not three; and the intensity stage, which the specification priced at two multiplication constraints, is the largest single item at 18 of 40 cells because a 57-bit gain shift needs four limbs and each clip decomposes its full input width. This chain was priced, not built; it is the first item of the research agenda.

VII. Fitting the Small-Satellite Envelope

A. Where the memory goes

The unsegmented prover peaks at 22.95 GiB (Keccak Merkle, degree-2 challenges) and 25.80 GiB (cubic challenges): trace generation 4.71, preprocessed commitment 8.44, main low-degree extension 16.17, permutation columns 20.81, quotient and FRI 22.95 GiB (Fig. 4). Memory tracks committed trace cells at about 40 GiB per 10^9 cells; commitment is 79% of the peak, and of the commitment 80% is the low-degree extension and 20% the Merkle tree. This is 3–50$\times$ the payload envelope.

B. What did not help

Three column-shaving changes were measured. The LogUp range table cut cells $2.70\times$ and time $1.45\times$, so it is a genuine lever. A radix-4 exact NTT probe cut cells $2.00\times$ and time $1.06\times$. An exact-NTT butterfly with four instead of sixteen columns gave no time win. Prover time follows committed columns and constraint degree, not trace width, and the range table had already taken most of that. Evaluating the preprocessed trace on the verifier instead of committing it would save 3%. A PCS substitution was measured directly:

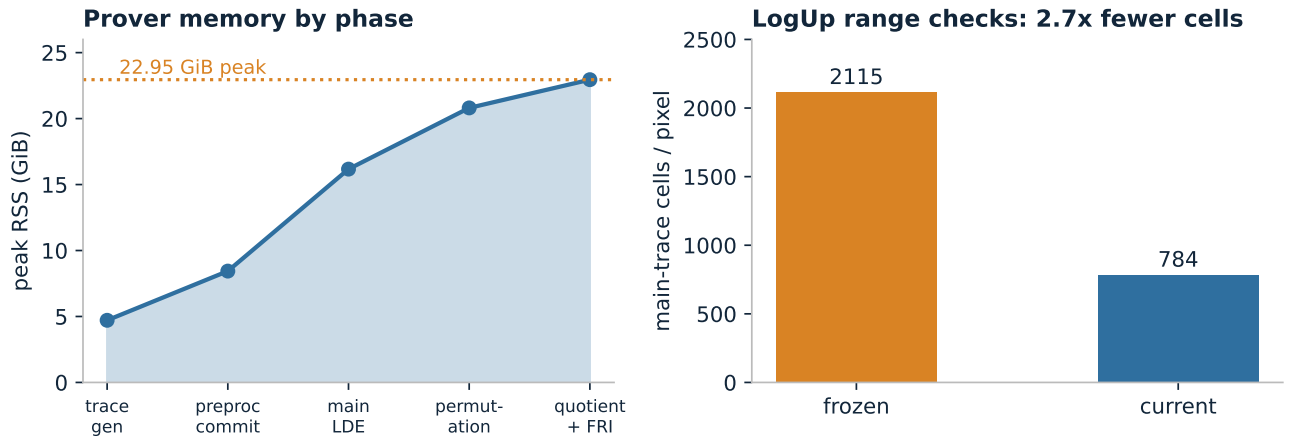


Fig. 4. Left: peak resident memory of the unsegmented prover by phase (59 tables, Keccak, degree-2 challenges). Right: main-trace cells per pixel before and after moving the range checks from bit columns to a shared LogUp table.

stock WHIR over the same 2^{29} cells, cubic extension, fold 8, at 100-bit Johnson soundness with zero grinding, peaks at 20.68 GiB for the commitment alone (57.5 B per cell), at parity with FRI's 22.95 GiB for the whole prover. Changing polynomial geometry does not change residency; reaching 1 GiB would need 1.8 B per cell, a 35–40× architectural change. It did produce an 847 KiB proof and it showed that the cubic extension reaches 100 bits without grinding, which became the security fix.

C. Segmentation with a joint lookup transcript

Cutting the table set into s segments and proving each separately fails naively because the lookup terminal of each segment is non-zero. The construction that works is a *segment set* (added to `p3-batch-stark` as a patch). In a first pass each segment commits its main trace and squeezes a digest from the transcript. A set-level frame then derives the shared LogUp challenges (α, β) and a batch digest from every segment's digest under a domain tag, so the challenges are still drawn after every trace is committed, which is what the LogUp soundness argument needs. In a second pass each segment observes the batch digest before its own constraint and FRI challenges, so that a segment proved against a different set fails its own opening proof and not merely the terminal sum; it returns its terminal unchecked, and the set verifier sums the terminals to zero. Bus identifiers are canonical across the set and each segment's local buses occupy a disjoint block, so one segment's imbalance cannot cancel another's; bus widths are declared because a width-2 tuple and a zero-padded width-3 tuple have equal fingerprints. The schedule differs from the standalone transcript by exactly one sample fewer and one observation more, and is written down once as data so that a second implementation (the recursive verifier) has one authoritative thing to agree with. The two-pass structure costs about 2× in time and is exactly the price of never holding more than one segment's prover data.

Measured (Table XII): 25.80 GiB at $s = 1$, 16.62 at 2, 8.51 at 4, **5.45 GiB at** $s = 8$ and **4.33 GiB at** $s = 16$ with balanced cuts. The floor is the largest single table (33 M cells): memory tracks the largest segment, not the total, and balanced cuts gain 1.97× over even cuts at $s = 16$. Going lower needs intra-table splitting. Time is bracketed at 2.0–3.03× by work counters, because total trace cells are identical at every s and the cost is protocol, not duplicated work. The price is bandwidth: eight segment proofs are 11.17 MB and sixteen are 17.84 MB, against 5.55 MB for the monolith and 2.10 MB of raw data. Segmentation solved memory and made bandwidth worse, which is what defines the next step.

D. Recursive aggregation

Recursion verifies a leaf proof inside a circuit and proves that circuit, so many segment proofs become one. Two frozen decisions collide: the security gate needs cubic challenge arithmetic, and the leaf's Merkle hash is a width-16 Goldilocks Poseidon2, and 3 does not divide 16. The resolution keeps both: the recursion circuit does its FRI arithmetic at degree 3 and hashes with a degree-1 width-16 Poseidon2 table, the only configuration that can reproduce the leaf's Merkle parents (widths 12 and 24 are different hashes and are inadmissible; the admissibility vector is one pinned arity-4 compression). The leaf proof is pruned (shared Merkle frontier); rather than implement the amortised frontier walk in-circuit, whose ceiling was computed at 1.12× on the Merkle term and 1.07–1.10× on the whole verifier, the prover emits the unpruned full openings ω_{rec} at proving time (2.31 MB, +3 MiB of RAM, time below noise) and feeds them to the unchanged per-query verifier circuit as untrusted witness; every path and opened value is re-checked against roots already in the transcript. The FRI transcript observes commitments, folding challenges, the final polynomial and grinding witnesses but never opening data, so the opening representation is transcript-invisible and ω_{rec} authenticates nothing by itself. Corrupting a sibling, an opened value, or swapping two query slots is rejected. The cost model is exact (Table XIII): 406 permutations per query, 78% of them leaf hashes that pruning cannot touch, 79,170 per leaf verification at 195 queries, 320 cells per permutation, and a verifier circuit of 11–16% of the leaf's own trace.

Four transcript pitfalls were encountered on the first real leaf and are generic to any recursive verifier over a Poseidon2-committed proof: a missing parameter set for the hash configuration; a third site demanding the "next-row" preprocessed opening; the challenger rate (8) confused with the leaf-hash rate (12) of the same permutation; and the prefix-free length tag bound at the wrong sponge lane. One permutation, two sponge splits, one configuration object. Each is now a millisecond unit test that fails before a full-leaf run is attempted. The segmented framing is mirrored in-circuit as public inputs (expected segment digest, batch digest, layout challenges); the sampled digest is constrained equal to the expected one, which is what makes the sample mean anything.

The measured results (Table XIV): one segment's verifier proves in 13 s and verifies in 41 ms, 2,251,836 bytes at 113.4 proven bits, already $1.074\times$ the raw acquisition. The flat aggregate of all eight segments, with the shared batch-digest target, the single h_{raw} carrier bound to the frozen value, native terminal binding, ordered coverage and the full-set closure $\sum_i \tau_i = 0$ enforced in-circuit, proves in 101 s, verifies in 45 ms, and is 2,722,924 bytes: $6.62\times$ smaller than eight separate recursive proofs and $1.298\times$ the raw acquisition, at a peak of 30.3 GiB. Memory grows with the number of complete leaf verifiers in one outer circuit (the ALU table is 72% of padded cells at $m = 8$), so the flat layout is a reference, not a flight architecture. Proof size is a step function of the outer ALU table's log-height, which is what a fixed FRI structure over a doubling trace predicts; two builds with identical geometry differ by 27 kB because the wire encoding is a varint, so smaller differences are noise. A tree of two 2-segment children under one parent measures lower (16.5 to 11.0 GiB at four segments) but the parent does not yet compose the aggregation summary across levels; that is unfinished. One implementation rule belongs in any recursion write-up: connecting two circuit targets aliases their witness slots, so an equality between two public producers must be expressed as a zero-constrained difference, or the outer proof fails at the lookup terminal with no indication of the cause.

E. The backend tax

Recursion requires the leaf to commit with Poseidon2 rather than Keccak. Bracketed on the eight-segment provenance relation (Keccak, Poseidon2, Keccak again as drift control), the Poseidon2 leaf costs $2.363\times$ the proving time (223.5 s against 94.5 s), uses slightly less memory (5.59 against 5.74 GiB) and produces 12.8% larger proofs (Table XV). The phase split attributes the whole penalty to the hash and commitment kernel (FRI phase $3.96\times$, quotient $0.99\times$). This tax is larger than the entire recursive verifier and is the first candidate for hardware. The system cost of a recursion-compatible design is therefore $T_{\mathrm{leaf}}^{\mathrm{Poseidon2}} + T_{\mathrm{recursion}}$, never the cheaper Keccak leaf plus a recursion increment.

VIII. Implementation and Results

A. Software

The prover is a Rust crate over a vendored Plonky3 [12] with three local patches (segment sets, recursion-witness emission, Merkle path restoration; 15 files) and a patched Plonky3-recursion (cubic ALU reduction, degree-1 width-16 hash table, legacy per-query FRI targets, segment framing, challenger lane fix; 35 files). The fixed-point reference and the simulator are Python with 447 unit tests; the AIR side has 33 Rust obligation tests (aggregate statement, summary composition, Merkle admissibility vector). The prover refuses to prove unless its trace reproduces the reference at seven checkpoints and the chain constants' digest matches. A thin verifier rebuilds every preprocessed table from the published arrays and the constants, never reading them from the proof. An end-to-end wrapper verifies the capture record against a pinned public key and only then invokes the prover with recomputation disabled. Every vendored change is captured as a patch against a pinned upstream commit and verified by reconstruction.

B. Correctness, security and evolution of the relation

Table XVI traces the relation from the frozen 58-table proof (bit-column range checks, 38.9 GiB, 3.57 MB) through the lookup table (784 cells per pixel, 22.95 GiB, 70.8 s, 2.46 MB, 11.8 ms to verify) and the cubic challenge field (113.4 proven bits at $1.20\times$ time and $1.12\times$ memory, output byte-identical) to the provenance-bound 62-table chain and the signed-capture demonstrator. Every configuration reproduces 241 detections, 18 components and 13 candidates and the same chain-constant digest. Table XVII gives the security sweep: prover time is flat in the query count (100.9 s at 200 queries against 100.8 s at 175), so queries are paid in proof bytes only, and the operating point of 195 queries was chosen where the bound saturates against the batch-combination term with 13 bits of margin over the 100-bit requirement rather than the frugal 170 that clears it by 0.9 bits. Verification is $6{,}000\times$ cheaper than proving and flat in scene size.

C. Provenance

The Poseidon2 sponge over the raw words (width 16, rate 12, capacity 4, overwrite mode, standard Goldilocks round constants, a six-element domain-separated header, three 16-bit words per element) costs 20.4 cells per pixel, 1.75% of the chain, and 29,128 permutations for the scene. It can be sharded across tables joined by a four-lane seam bus; the digest is identical at 1, 2, 4 and 8 shards, and a cut that would start a shard on the partial tail block is rejected at construction. The raw words reach the sponge on the same bus that feeds the signal processing, so the lemma $D_{\mathrm{hash}}[a] = \mathrm{u16}(D_{\mathrm{SAR}}[a])$ is enforced by multiset equality with a signed-to-u16 adapter of six sign columns per row; the bus pins the value, a 15-bit lookup pins the sign. The acceptance test for provenance is a single flip: the raw substitution case moved from (verify, exact) = (True, False) to (False, False) with the honest baseline still accepted. Table IV places this among the alternatives.

D. Attack batteries

Table XVIII lists 26 hostile cases in the chain, provenance, segment-set and recursive verifier batteries, plus 8 capture-record and fail-closed demonstrator cases. Each battery is read

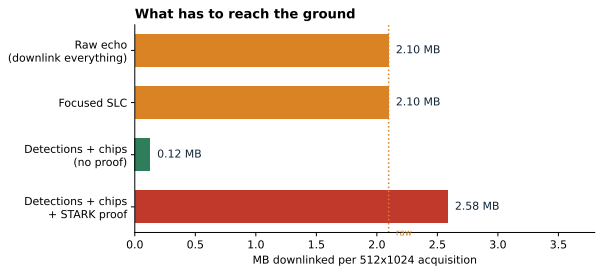


Fig. 5. Bytes to the ground per 512×1024 acquisition at the Keccak, degree-2 operating point of 24 August: the products are $17\times$ smaller than the raw echo, and the proof puts them back above it. The cubic and recursive operating points in Table XX do not change the picture.

honest-baseline first (a battery in which everything is rejected is indistinguishable from a verifier that rejects everything). Cases 2, 3, 4 and 7 keep the output exact because they corrupt an intermediate or a published value while the internal stages still reproduce the reference; cases 5, 6, 8 and 9 change the computation itself. The four framing attacks on the recursive verifier reject at four different witness sites, which is the evidence that four mechanisms are at work rather than one reported four times; the association attack, in which every value is individually legitimate and only the pairing is wrong, is the sharpest. The demonstrator battery re-hashes every mutated artefact against the honest one before reading a verdict, so that a mutation that failed to apply cannot be reported as rejected.

E. Downlink

Table XX and Fig. 5 give the downlink accounting, the one result that does not yet meet its target. Against the 2,097,152-byte raw acquisition, the products alone are 6%; the unsegmented proof at 58.8 bits is $1.17\times$; the eight segmented leaf proofs are $6.7\times$; one recursively proved segment is $1.07\times$; the flat aggregate of all eight is $1.298\times$ at 113.4 bits and $1.143\times$ at the 100-bit minimum of 169 outer queries. Proof size is linear in the outer query count, $|\pi| = 278{,}704 + 12{,}534.5\,q$ bytes, and reaches the raw size only at $q = 145$, 24 queries below the 100-bit floor; the wire format is at its floor (field elements are already canonical 8-byte, and a fixed-width encoding is larger). At this scene and these FRI parameters the proof cannot get below the raw data while meeting 100 proven bits.

F. End-to-end demonstrator

The demonstrator (Table XXI) runs capture-record verification against a pinned key, proves the chain with h_{raw} taken from the record and no fallback, and verifies: 192.8 s to prove including trace generation and recursion instrumentation, 92 ms to verify, 12 of 12 regression cases. A graphical front end (Fig. 6) draws the intensity image, CFAR score, detection mask and chips only from arrays whose digests the proof binds; every other panel is labelled NOT PROVEN, the attack panel reports the battery only for the scene on screen, and changing the scene drops the proof. Nothing shown on screen is unproven.

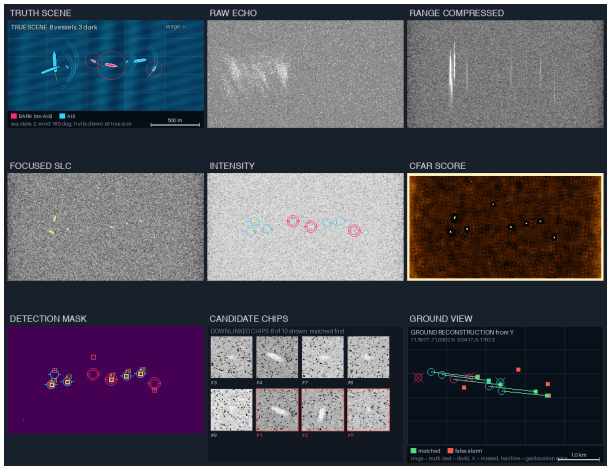


Fig. 6. The nine panels of the demonstrator front end, rendered by the application's own headless renderer for its self-check scene (seed 7, eight vessels, 512^2). In the live dashboard each panel carries a PROVEN or NOT PROVEN caption from the proof manifest, and the intensity, CFAR score, mask and chips are drawn only from the published arrays whose digests the proof binds; a screenshot of the live window is to be substituted here.

IX. Assessment

A. What is established

Judged against the objective of Section I, the following is established on the simulated scene and is, to our knowledge, the first time each has been shown for a complete SAR detection chain. The chain is proven bit-exactly, from raw echo to published chips, with the dataflow between 62 tables enforced by lookup arguments rather than assumed. Soundness is 113 proven bits, with the estimator run on the real constraint set rather than on a model of it. The acquisition is bound into the statement by a hash fixed at capture, and the self-consistent substitution attack, the one a signature-based design cannot see, is rejected. The working set fits a 4–8 GiB accelerator envelope through segmentation with no new prover architecture. Recursion runs on real SAR proofs and compresses eight leaves into one externally verified proof at unchanged security. Verification is milliseconds and flat in scene size. Two whole alternative routes, the Boolean circuit and the multilinear commitment as a memory fix, were measured and closed rather than argued away, which narrows the design space for whoever comes next. And the cost structure is now understood in detail: roundings and wiring are 77% of the cells, hashing is the whole of the recursion backend tax, and both are addressable.

B. What remains open, and the lever for each

Four gaps separate this from a flight system, and each has a measured cause.

Downlink. No measured configuration transmits fewer bytes than the raw acquisition it certifies; the best is $1.14\times$ before products. Segmentation and recursion together deliver a $6.6\times$ reduction over eight leaf proofs, but the outer proof's fixed cost at 100 bits is above raw at this scene. The ratio moves in the right direction with scene size, since the outer proof is roughly fixed while raw grows with pixels, but the scaling

law was not measured and we do not extrapolate beyond a factor of two from a measured point. The levers are the scene-size measurement itself and the root relation of a hierarchical aggregate.

Throughput. The Poseidon2 leaf prover produces 2 MiB of certified acquisition per 223.5 s on 18 laptop cores; repeated naively that is 32 hours per GiB of leaf proving before aggregation, halos or a flight processor's derating. The levers are the certified relation (a counted $10.9\times$), a sumcheck front end for the linear stages (a counted $80\times$ on those terms) and a hardware hash kernel (the $2.36\times$ backend tax is entirely hashing). The recorded GPU precedent for STARK proving is $22.8\times$; the required factor is in the tens, which is a conventional accelerator problem, not a proof-system impossibility.

Composition. The hierarchical aggregate lowers memory but does not yet bind the per-level summary, so it is a resource result awaiting its cryptographic half; the summary relation is specified and unit-tested, and its in-circuit composition is engineering.

Capture boundary. The capture component is a signed record produced by a separate process with a development key, and the signature is Ed25519. The proof system is transparent and hash-based; the whole system becomes post-quantum when the capture signature is.

C. Deployment paths by requirement

The right architecture depends on the customer's required statement rather than on the proof backend (Table XXII). The succinct-proof route serves the always-on, capture-only-trust requirement and is the one measured here. Where a customer can verify explicitly requested regions, the same capture root supports authenticated raw support with ground replay; where a customer accepts a larger trusted computing base, a protected native chain removes almost all proving work. The three paths share the capture root, the canonical serialisation, the fixed-point reference and the hostile-test discipline built here, so investment in any of them transfers.

X. A Roadmap from the Laboratory to Flight

The measurements above define the path. We group it into the work that changes the cost structure and the work that moves the system on board, in the order we would take it.

A. Reducing prover cost

1) **Adopt the exact-convolution relation.** Exact integer convolution by NTT with one requantization removes six roundings per butterfly: 40 against 436 cells per pixel (counted), $10.9\times$, and 29 dB more accurate. It is a decision about the certified DSP chain, not about the proof system, and it is the largest single lever available. The reference implementation already contains the stage.
2) **Prove the linear stages by sumcheck.** Range and azimuth compression as $y = Mx$ verified against committed columns contribute no cells of their own; counted at $80\times$ on the convolution terms and robust across two to six cells per tap. This requires a multilinear commitment with arbitrary-point opening (WHIR, BaseFold or Ligerito) over the same Goldilocks primitives; the univariate AIR stack cannot host it, so this is an architectural step rather than a tuning one. The lookup machinery for the nonlinear residue carries over unchanged.
3) **Remove the lookup helper columns.** LogUp-GKR commits only a multiplicities column and targets the 32.7% of cells that are wiring; the experiment is pre-registered with a $\geq 15\%$ prover-time threshold. Intra-table splitting is the route below the 4.33 GiB segmentation floor.
4) **Complete hierarchical aggregation.** Bind a fixed summary (coverage interval, segment-set identity, carrier count, authenticated raw root, cross-segment terminal, output binding) into each child and constrain it in the parent, with real-proof rejection tests for substituted, repeated, cross-acquisition and proper-prefix children; then measure leaf plus hierarchy in one run. This is the step that turns the tree's memory result into the downlink result.
5) **Measure the scene-size law.** Double the scene from 512×1024 one step at a time, repeating the segmentation and aggregation measurements at each step, until the outer proof crosses below the raw acquisition. The crossover is expected because the outer proof is nearly fixed in size, and finding it is the first quantitative bandwidth claim this system can make.

B. From laboratory to on-board testing

6) **Measure the two flight-core constants.** Cycles per Goldilocks multiply and per hashed byte on the flight-class core collapse every banded estimate in this paper. A Cortex-A53 development board suffices, and the ARMv8 cryptographic extension is worth a factor of five on the hash term, which makes it a procurement decision.
7) **Port the prover to a flight-representative SoC.** A Zynq UltraScale+ class device with ARM cores and fabric is the natural target: NTT and Poseidon2 Merkle kernels on fabric, control and lookup logic on the cores. The measured phase split says where the fabric effort goes (hashing and commitment first). Report per-scene wall time, peak memory, joules and storage traffic on that board, not on a laptop.
8) **Build the capture component.** A minimal fixed Poseidon2 capture core on the sensor-side FPGA with a monotonic counter, DMA-protected hashing of the sample stream, the frozen record schema (scheme version, instance identifier, digest, geometry, acquisition identifier) and a post-quantum signature profile. This is the smallest piece of trusted hardware in the system and the one on which the whole guarantee rests.
9) **Integrate with a native SAR processor.** On-board focusing and detection already exist at prototype level on FPGA/ARM hardware [3]; the prover consumes the same fixed-point intermediates. Running the two side by side, with the native processor producing the

product and the prover producing the certificate for it, is the configuration to benchmark for continuous service, including the thermal envelope of a 6U or 12U bus.

10) **Real data.** Sentinel-1-class Level-0 bursts with their BAQ decoding and calibration dependencies, C-band geometry, coastal and open-sea scenes. Bit-exact replay against an independent focuser is the acceptance test; image-quality metrics are reported separately.
11) **Exploit the self-checking property under radiation.** A single-event upset in the prover cannot produce an accepted wrong product; it produces a rejected proof. The detector and the prover therefore need different fault-tolerance treatment: the prover can run on commercial parts with retry, and only the capture component needs hardening. Quantifying the retry rate under representative fault injection is a cheap experiment with a large effect on the parts budget.
12) **Ground segment.** Package the verifier as an independent executable with no access to the on-board witness, pin the capture keys, and run it on received artefacts only. The verifier is milliseconds; the engineering is key custody and the reconstruction of the statement from public inputs.
13) **Staged demonstration.** (a) Hardware-in-the-loop on the bench with recorded raw data and the flight SoC, all 26 hostile cases rerun on the flight binary; (b) an engineering model on a CubeSat avionics stack in tip-and-cue mode, where raw data is retained and the certificate can be audited against it; (c) a hosted-payload flight experiment on a SAR smallsat in always-on mode, where the proof is the only evidence of correct processing. Each stage has the same acceptance test: the honest product verifies, and every manipulation in Table XVIII is rejected.

The gating question for the whole path is quantitative and cheap to answer first: does the certified product, with its proof, save bytes on a real packetized acquisition at a scene size the flight processor can handle within its duty cycle? Steps 1, 2 and 5 answer it, and every subsequent step assumes a yes.

XI. Conclusion

A satellite can prove, with a transparent post-quantum proof at 113 proven bits, that a set of ship detections and image chips was computed by the prescribed fixed-point SAR chain from an acquisition whose digest was fixed at capture, and it can do so within a 4–8 GiB working set, with a verifier that runs in milliseconds. That is the integrity guarantee that always-on maritime surveillance from a compromised platform requires, and it is now demonstrated end to end on a simulated scene, with every attack in a 26-case battery rejected. The measurements that establish it are in Tables I to XXII. What remains is cost: the proof does not yet undercut the raw bytes, and proving does not yet keep mission cadence. Both are quantified, and the two levers that dominate them, an exact-convolution relation worth $10.9\times$ and a sumcheck front end worth $80\times$ on the linear stages, are counted, not built. They are where the work should go next, alongside the port to a flight-representative processor that turns the laboratory bands of this paper into flight numbers.

References

[1] European Space Agency, "Sentinel-1 data-flow budget," https://eof.esa.int/docs/api/eof-doc/get-html-sph/58130736, 2026.
[2] DLR, "S4Pro on-board payload processing requirements, d1.3," https://elib.dlr.de/130314/, 2019, vERIFY year.
[3] H. Breit *et al.*, "Demonstrating a SAR satellite onboard processing," DLR, https://elib.dlr.de/188283/, 2022, eO-ALERT prototype; VERIFY.
[4] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote ATtestation proceduress (RATS) architecture," RFC 9334, 2023.
[5] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, "Delegating computation: interactive proofs for muggles," in *STOC*, 2008.
[6] C. Lund, L. Fortnow, H. Karloff, and N. Nisan, "Algebraic methods for interactive proof systems," in *FOCS*, 1990.
[7] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," Cryptology ePrint Archive, Paper 2018/046, 2018.
[8] U. Haböck, "Multivariate lookups based on logarithmic derivatives," Cryptology ePrint Archive, Paper 2022/1530, 2022.
[9] S. Papini and U. Haböck, "Improving logarithmic derivative lookups using GKR," Cryptology ePrint Archive, Paper 2023/1284, 2023.
[10] Succinct Labs, B. Bünz, and W. Wang, "Flock: fast batch boolean R1CS proving over GF(2)," Cryptology ePrint Archive, Paper 2026/1329; https://github.com/succinctlabs/flock, 2026, vERIFY title and author list.
[11] A. Novakovic and G. A. Gruen, "Ligerito: a small and concretely fast polynomial commitment scheme," Cryptology ePrint Archive, 2025, vERIFY authors and paper number.
[12] Polygon Zero and contributors, "Plonky3: a toolkit for polynomial iops," https://github.com/Plonky3/Plonky3, 2026, vendored at commit 3c902a9 (2026-08-20); Plonky3-recursion at b363397 (2026-07-06).
[13] L. Grassi, D. Khovratovich, and M. Schofnegger, "Poseidon2: a faster version of the Poseidon hash function," Cryptology ePrint Archive, Paper 2023/323, 2023.
[14] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Fast Reed–Solomon interactive oracle proofs of proximity," in *ICALP*, 2018.
[15] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, "WHIR: Reed–Solomon proximity testing with super-fast verification," Cryptology ePrint Archive, Paper 2024/1586, 2024.
[16] H. Zeilberger, B. Chen, and B. Fisch, "BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes," Cryptology ePrint Archive, Paper 2023/1705, 2023.
[17] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, "Brakedown: linear-time and field-agnostic SNARKs for R1CS," in *CRYPTO*, 2023.
[18] E. Crites and A. Stewart, "On Reed–Solomon proximity gaps conjectures," Cryptology ePrint Archive, Paper 2025/2046, 2025, disproves capacity-level correlated-agreement and list-decodability conjectures; VERIFY title.
[19] B. E. Diamond and J. Posen, "A crack in the firmament: restoring soundness of the Orion proof system and more," Cryptology ePrint Archive, Paper 2024/1164, 2024, vERIFY author list.
[20] D. Khovratovich, R. D. Rothblum, and L. Soukhanov, "How to prove false statements: practical attacks on Fiat–Shamir," Cryptology ePrint Archive, Paper 2025/118, 2025.
[21] A. K. Lenstra, H. W. Lenstra, and L. Lovász, "Factoring polynomials with rational coefficients," Mathematische Annalen 261, 1982.
[22] H. Cohen, "A course in computational algebraic number theory," Springer GTM 138, 1993.
[23] Trusted Computing Group, "Trusted platform module library specification, family 2.0," Version 185, 2026.
[24] ——, "DICE attestation architecture," 2021.
[25] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," RFC 9162, 2021.
[26] T. Liu, X. Xie, and Y. Zhang, "zkCNN: zero knowledge proofs for convolutional neural network predictions and accuracy," Cryptology ePrint Archive, Paper 2021/673; ACM CCS 2021, 2021.
[27] I. G. Cumming and F. H. Wong, *Digital processing of synthetic aperture radar data: algorithms and implementation*. Artech House, 2005.
[28] European Space Agency, "Sentinel-1 level-1 detailed algorithm definition," https://sentinels.copernicus.eu/documents/247904/1877131/Sentinel-1-Level-1-Detailed-Algorithm-Definition, 2026.
[29] Polyhedra Network, "Expander: a GKR-based proof system, v1.2.0," https://github.com/PolyhedraZK/Expander, 2025.